



Computing Join Queries with Functional Dependencies

Mahmoud Abo Khamis
LogicBlox Inc. and
SUNY Buffalo

Hung Q. Ngo
LogicBlox Inc. and
SUNY Buffalo

Dan Suciu
LogicBlox Inc. and
University of Washington

ABSTRACT

Recently, Gottlob, Lee, Valiant, and Valiant (GLVV) presented an output size bound for join queries with functional dependencies (FD), based on a linear program on polymatroids. GLVV bound strictly generalizes the bound of Atserias, Grohe and Marx (AGM) for queries with no FD, in which case there are known algorithms running within the AGM-bound and thus are worst-case optimal.

A main result of this paper is an algorithm for computing join queries with FDs, running within GLVV bound up to a poly-log factor. In particular, our algorithm is worst-case optimal for any query where the GLVV bound is tight. As an unexpected by-product, our algorithm manages to solve a harder problem, where (some) input relations may have prescribed maximum degree bounds, of which *both* functional dependencies and cardinality bounds are special cases.

We extend Gottlob et al. framework by replacing all variable subsets with the lattice of closed sets (under the given FDs). This gives us new insights into the structure of the worst-case bound and worst-case instances. While it is still open whether GLVV bound is tight in general, we show that it is tight on distributive lattices and some other simple lattices. Distributive lattices capture a strict superset of queries with no FD and with simple FDs. We also present two simpler algorithms which are also worst-case optimal on distributive lattices within a single-log factor, but they do not match GLVV bound on a general lattice. Our algorithms are designed based on a novel principle: we turn a proof of a polymatroid-based output size bound into an algorithm.

Keywords

Worst-case optimal; Join algorithm; Functional dependency; Entropy; Polymatroid

1. INTRODUCTION

Several results published in the last ten years or so have lead to tight worst-case output size bounds (*AGM-bound* [5, 13]) and the development of a new class of query processing

algorithms running within the bound's time budget [18, 19, 23]. These new *worst-case optimal* algorithms are quite different from traditional query plans, in the sense that they no longer compute one pairwise join at a time, but instead process the query globally. The runtime is bounded by $O(N^{\rho^*})$, where N is the size of the database, while ρ^* is the value of the optimal fractional edge cover of the query. For example, they compute the query $Q(x, y, z) :- R(x, y), S(y, z), T(z, x)$ in time $O(N^{3/2})$, while any traditional query plan requires worst-case time $\Omega(N^2)$ [19].

While the vast majority of database engines today still rely on traditional query plans, new, complex data analytics engines increasingly switch to worst-case optimal algorithms: LogicBlox's engine [4] is built on a worst-case optimal algorithm called *LeapFrog Triejoin* [23] (LFTJ), the *Myria* data analytics platform supports a variant of LFTJ [8], and the increased role of SIMD instructions in modern processors also favors this new class of join algorithms [1].

When there are functional dependencies (FDs), however, the AGM-bound is no-longer tight. Gottlob, Lee, Valiant, and Valiant [12] initiated the study of the worst-case output size of a query in the presence of FDs and described an upper bound (reviewed in Sec.2), called the *GLVV-bound* henceforth. It remains open whether the GLVV-bound is tight; at present, it is the best known upper bound for the output size of a join query with FDs. A recent result by Gogacz and Toruńczyk [11] proved a weaker statement, namely that if we replace the polymatroidal constraints in the GLVV-bound with entropic constraints (of which there are infinitely many), then the bound is tight, albeit uncomputable.

Our paper proposes an approach to studying join queries in the presence of FDs by using lattice theory. The GLVV-bound is reformulated with a very simple linear program on the lattices' polymatroids, and output size bounds are obtained by studying formal inequalities satisfied by all these polymatroids. This approach allows us to derive both new lower and upper bounds on the worst-case output size. Furthermore, we present an algorithm which runs in time proportional to the GLVV-bound, within a polylogarithmic factor. We also describe two special cases where the polylogarithmic factor is reduced to a single log. Before discussing our results, we explain their significance.

1.1 Motivation

User-Defined Functions. UDF's, or interpreted predicates, significantly affect the runtime of a query. Consider:

$$Q(x, y, z, u) :- R(x, y), S(y, z), T(z, u), u = f(x, z), x = g(y, u). \quad (1)$$

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

PODS'16, June 26-July 01, 2016, San Francisco, CA, USA

© 2016 ACM. ISBN 978-1-4503-4191-2/16/06...\$15.00

DOI: <http://dx.doi.org/10.1145/2902251.2902289>

The predicates $u = f(x, z)$ and $x = g(y, u)$ represent two user-defined functions, f and g ; for example $f(x, z)$ could be $x + z$, or the concatenation of two strings x and z . Any UDF can be modeled as a relation with a primary key, for example the function f can be viewed as a relation $F(x, z, u)$ of cardinality N^2 (with one entry for every x, z pair in their effective domains) satisfying the FD $xz \rightarrow u$; similarly g introduces the FD $yu \rightarrow x$. The addition of these two FD's significantly affects the output size and the query evaluation complexity: if we computed the intermediate query $R(x, y), S(y, z), T(z, u)$, and then applied the two predicates $u = f(x, z)$ and $x = g(y, u)$, then the runtime can be as high as N^2 because the size of the intermediate result is N^2 for $R = \{(i, 1) \mid i \in [N]\}$, $S = \{(1, 1)\}$, $T = \{(1, i) \mid i \in [N]\}$. However, as we will show, the GLVV-bound is $|Q| \leq N^{3/2}$. Our new algorithms run in $\tilde{O}(N^{3/2})$; and as we shall see this is worst-case optimal.

A related problem is querying relations with restricted access patterns [7]. In that setting, some of the relations in the database can only be read by providing the values of one or more attributes. As shown above, a user-defined function $f(x, z)$ can be modeled as an infinite relation $F(u, x, z) \equiv (u = f(x, z))$ with the restriction that F can be accessed only by providing inputs for the variables x and z . The work on querying under restricted access patterns has focused on the answerability question (whether a query can or cannot be answered). Our work extends that, by finding algorithms for answering the query within the GLVV-bound.

Known Frequencies. Systems often know an upper bound on the frequencies (or degrees) in the database. For example, consider the triangle query above, and assume that the binary graph defined by the relation $R(x, y)$ has a bounded degree: all outdegrees are $\deg(x) \leq d_1$ and all indegrees are $\deg(y) \leq d_2$. One can model this scenario by introducing an artificial color c_1 on outgoing edges, and a color c_2 for incoming edges of R :

$$Q(x, y, z, c_1, c_2) :- R(x, c_1, c_2, y), S(y, z), T(z, x), C_1(c_1), \\ C_2(c_2), xc_1 \rightarrow y, yc_2 \rightarrow x, xy \rightarrow c_1c_2 \quad (2)$$

where $|R| = |S| = |T| = N$ and $|C_1| = d_1, |C_2| = d_2$. Thus, each outgoing edge from x is colored with some distinct color c_1 , and similarly each incoming edge to y is colored with some distinct color c_2 . The new predicates C_1, C_2 limit the number of colors to d_1, d_2 respectively. We can show that the worst-case query output decreases from $N^{3/2}$ to $\min(N^{3/2}, Nd_1, Nd_2)$ [2]. Alternatively, one can use the linear program and algorithm in Sec 5.2 to capture queries with known maximum degree bounds. We note that, by adapting AYZ algorithm [3], degree information was used in [15] to obtain a sub-quadratic time algorithm for join on a special class of query graphs called *1-series-parallel graphs*. Their approach also uses a linear program similar to the one in Sec 5.2, minus the sub-modularity constraints.

1.2 Overview of the Results

Grohe and Marx [13] and later Atserias, Grohe and Marx [5] derived an elegant tight upper bound on the output size of a join query: $\prod_{j=1}^m |R_j|^{w_j}$, where $R_1, \dots, R_m$ are the input relations to the query, and $(w_j)_{j=1}^m$ is any fractional edge cover of the query's hypergraph. This bound is known today as the AGM bound. They proved that the bound is tight by describing a simple database instance for any query, such that the query's output matches the bound. In that instance,

every relation is a cross product of sets, one set per variable; we call it a *product instance*.

An obvious open question was whether a query Q can be evaluated on any database instance D in time that is no larger than the AGM bound of Q on databases with the same cardinalities as D ; such an algorithm is called *worst-case optimal*. Ngo, Porat, Ré, and Rudra [18] described the first worst-case optimal algorithm; later Veldhuizen [23] proved that LFTJ, an algorithm already implemented at LogicBlox earlier, is also worst-case optimal. A survey and unification of these two algorithms can be found in [19].

Neither the AGM-bound nor the associated algorithms analytically exploit FDs in the database.¹ Such FDs can provably reduce the worst-case output of a query, but the upper bound and algorithms mentioned above cannot use this information, and instead treat the query by ignoring the FDs. Gottlob et al. [12] studied the upper bound of the query size in the presence of FDs, and established two classes of results. The first was a characterization of this bound in the case when the FD's are restricted to simple keys; as we will show, this case can be solved entirely using the AGM bound by simply replacing each relation with its closure. The second was a novel approach to reasoning about the worst-case output of a query, using information theory. They viewed the query output as a multivariate probability space, and introduced two constraints on the marginal entropies: a cardinality constraint for each input relation R , stating that the entropy of its variables cannot exceed the uniform entropy $H(\text{vars}(R)) \leq \log_2 |R|$, and one constraint for each FD $X \rightarrow Y$, stating $H(XY) = H(X)$. (Note that X and Y are sets of variables.) The largest answer to the query Q is then given by the largest possible value of $2^{H(\text{vars}(Q))}$, over all choices of entropic functions H that satisfy these constraints. Characterizing the space of all entropic functions H is a long-standing open problem in information theory [24]; to circumvent that, they relax the function H by allowing it to be any function that satisfies the *Shannon inequalities* [24]. Such a function is called a *polymatroid* in the literature, and we denote it with lower case h to distinguish it from entropic functions H . Thus, the problem in [12] can be stated equivalently as: find the maximum value $h(\text{vars}(Q))$ where h ranges over all polymatroids satisfying the given constraints.

In this paper, we continue the study of query evaluation under general FDs. We establish both bounds and algorithms. Our novelty is to model FDs as a lattice $\mathbf{L}$ of the closed sets of attributes, and to study polymatroids on lattices. The function h is now any non-negative and lattice-submodular function² that satisfies all cardinality constraints. FD constraints are enforced *automatically* by the lattice structure, and the upper bound on the query size is $2^{\max h(\text{vars}(Q))}$. When there are no FDs, the lattice is a Boolean algebra.

Our first question is whether the elegant AGM bound and worst-case product instance carry over to arbitrary FDs. We answer this question completely, by proving that both sides of the AGM bound hold iff the lattice has a special structure,

¹Algorithmically, LFTJ handles FDs by binding variables at the earliest trie level at which they are functionally determined. For example, in $R(x, y), S(y, z), T(z, u), u = f(x, z), x = g(y, u)$ with a key order $[x, y, z, u]$, whenever z was bound the value of u would be immediately computed by $u = f(x, z)$.

²I.e. $h(X) + h(Y) \geq h(X \vee Y) + h(X \wedge Y)$

which we call a *normal lattice*. Both upper and lower bounds require minor extensions to be applicable to normal lattices. The standard AGM upper bound is given in terms of fractional edge covers of the query's hypergraph, but in a normal lattice one needs to consider a dual hypergraph, whose nodes are $\mathbf{L}$'s co-atoms. In a Boolean algebra, these two hypergraphs are isomorphic, but in a general lattice they can be significantly different. The notion of normal lattice seems novel, and strictly includes all distributive lattices, which in turn include all lattices corresponding to simple FDs (each FD is of the form $a \rightarrow b$, where a, b are attributes). The second minor change is that one needs to allow for a slight generalization of product instances, to what we call *quasi-product instances*. Importantly, both these properties fail on non-normal lattices; in particular, on non-normal lattices worst-case instances are not quasi-product instances.

Next, we examine algorithms whose runtime is bounded by the GLVV bound. We propose a novel methodology for designing such algorithms, starting from the observation that such an algorithm must provide a proof of the query's upper bound, equivalently, a proof of an inequality of the form $\sum_j w_j h(\text{vars}(R_j)) \geq h(\text{vars}(Q))$, where $R_1, R_2, \dots$ are the input relations. In the case of a Boolean algebra, Shearer's lemma [9] is of this form; in a normal lattice this corresponds to a fractional edge cover of the co-atomic hypergraph; and, for a general lattice it is a general inequality. A key motivation behind NPRR [18] was to prove inequalities algorithmically. This paper completes the cycle by proceeding in the opposite direction: given a proof method for such inequalities, we design algorithms whose steps correspond to the proof steps. We design three such algorithms, corresponding to three methods for proving the above type of inequalities.

The first algorithm called the *chain algorithm* runs within the *chain bound*; the proof technique is adapted from Radhakrishnan's proof [20] of Shearer's lemma to general lattices. Both the chain bound and algorithm strictly generalize AGM-bound and worst-case optimal algorithms for join queries *without* FDs. The second algorithm, called the *sub-modular algorithm*, runs within the *sub-modularity bound*; the proof technique is that of Balister and Bollobás's [6]. The third algorithm, called the *conditional sub-modularity algorithm* (CSMA), runs within the general *GLVV-bound*, up to a polylogarithmic factor; the proof technique is our own, based on linear programming duality. As a bonus, CSMA can be used straightforwardly to handle input relations with known maximum degree bounds, which is a strict generalization of the problem of computing join queries with FDs. While GLVV bound is tighter than the other two in general lattices, we show that in distributive lattices all of them are tight, and thus our algorithms are worst-case optimal.

Outline. The paper is organized as follows. Background material is reviewed in Sec. 2, and basic definitions for our lattice-based approach are given in Sec. 3. We describe the main result on normal lattices in Sec. 4, then present two of our algorithms and bounds in Sec. 5. Due to space limitation, some details were left out of this extended abstract, including the sub-modularity bound and algorithm. Interested readers can find them in the full version [2].

2. NOTATIONS AND PRIOR RESULTS

For any positive integer n , $[n]$ denotes the set $\{1, \dots, n\}$. All log in the paper are of base 2. We fix a relational schema

$\mathbf{R} = \{R_1, \dots, R_m\}$ whose attributes belong to a set of attributes $\mathbf{X} = \{x_1, \dots, x_k\}$. We refer to $x_i \in \mathbf{X}$ interchangeably as an *attribute* or a *variable*; similarly we refer to R_j as a *relation*, or an *input*. We use lower case letters $x \in \mathbf{X}$ to denote single variables, and upper case letters $X \subseteq \mathbf{X}$ to denote sets of variables. The *domain* of variable x is denoted by $\text{Dom}(x)$. For each relation R_j we denote $\text{vars}(R_j) \subseteq \mathbf{X}$ its set of attributes. Abusing notation, we blur the distinction between R_j and $\text{vars}(R_j)$, writing $R_j \subseteq \mathbf{X}$. We consider full conjunctive queries without self-joins:

$$Q(x_1, \dots, x_k) :- R_1(\text{vars}(R_1)), \dots, R_m(\text{vars}(R_m)) \quad (3)$$

We will drop variables from the head, since it is understood that all variables need to be listed.

A database instance D consists of one relational instance R_j^D for each relation symbol; we denote $N_j = |R_j^D|$, $N = |D| = \sum_j N_j$, and use lower case for logs, $n_j = \log_2 N_j$. We denote Q^D the answer to Q on the database instance D . When D is implicit from context, we will simply write Q, R_j instead of Q^D, R_j^D . A *product database instance* is a database instance such that $R_j^D = \prod_{x_i \in R_j} \text{Dom}(x_i)$ for $j \in [m]$; the query answer on a product database is the cross product of all domains, $Q^D = \prod_{i=1}^k \text{Dom}(x_i)$.

The AGM Bound. A series of results over the last ten years [5, 13, 18, 19, 23] has established tight connections between the maximum output size of a query and the runtime of a query evaluation algorithm. The *query hypergraph* of a query Q is $H_Q = (\mathbf{X}, \mathbf{R})$; its nodes are the variables and its hyperedges are the input relations (where each $R_j \in \mathbf{R}$ is viewed as a set of variables). Consider the following two linear programs (LP's):

$$\begin{array}{ll} \text{(Weighted) Edge Cover} & \text{(Weighted) Ver. Packing} \\ \text{minimize } \sum_{j=1}^m w_j n_j & \text{maximize } \sum_{i=1}^k v_i \\ \sum_{j: x_i \in R_j} w_j \geq 1, \forall i \in [k] & \sum_{i: x_i \in R_j} v_i \leq n_j, \forall j \in [m] \\ w_j \geq 0, \forall j \in [m] & v_i \geq 0, \forall i \in [k] \end{array}$$

We call a feasible solution to the first LP a *weighted fractional edge cover*, and to the second a *weighted fractional vertex packing*. The traditional (unweighted) notions correspond to $n_j = 1, \forall j$.

Theorem 2.1 (AGM bound). [5, 13] (1) Let $(w_j)_{j=1}^m$ be a fractional edge cover. Then, for any input database D s.t. $|R_j^D| \leq N_j$ for all $j \in [m]$, the output size of Q is bounded by $2^{\sum_j w_j n_j}$. In other words, $|Q^D| \leq \prod_j N_j^{w_j}$. (2) Let $v_i, i \in [k]$ be a fractional vertex packing, and let D be the product database instance where $D_i = [2^{v_i}]$. Then, $|Q^D| = \prod_i 2^{v_i}$.

(In the statement above, we ignore the issue of integrality of the v_i for the sake of clarity. There is a bit of loss when v_i are not integers, but this fact does not affect the asymptotics of the lowerbound [5].) Let $(w_j^*)_{j=1}^m$ be the optimal edge cover. The *AGM-bound* of the query Q is $\text{AGM}(Q, (N_j)_{j=1}^m) \stackrel{\text{def}}{=} \prod_j N_j^{w_j^*}$, or simply $\text{AGM}(Q)$ when the cardinalities $(N_j)_{j=1}^m$ are clear from the context. The query's output size is always $\leq \text{AGM}(Q)$, and this bound is tight, because on the product database described above, the output size is $\text{AGM}(Q)$. Several query evaluation algorithms

have been described in the literature with runtime³ $\tilde{O}(N + \text{AGM}(Q))$: NPRR [18], LFTJ [23], Generic-join [19].

Functional Dependencies. A query with functional dependencies is a pair (Q, FD) , where Q is a query and FD is a set of *functional dependencies* (fd), which are expressions of the form $U \rightarrow V$ where $U, V \subseteq \mathbf{X}$. An fd can be defined either by some input relation R_j , in which case we call it *guarded* (in particular $U, V \subseteq R_j$), or can be defined by a UDF (as we saw in Sec. 1.1), and then we call it *unguarded*. A *simple* fd is of the form $u \rightarrow v$ where both u, v are variables, and a *simple key* is a simple fd guarded in some R_j s.t. u is a key for R_j .

Gottlob et al. [12] studied output size bounds in the presence of FDs. Their bound is defined only in terms of the maximum cardinality, $\max_j N_j$, but in this paper we slightly generalize the discussion to all cardinalities $(N_j)_j$. The key technique introduced in [12] is to use information theory to upper bound the size of the query, as reviewed next.

Let Q^D be the query answer over some instance D . Define a probability distribution over $\prod_{i=1}^k \text{Dom}(x_i)$ by randomly drawing one tuple from Q^D with probability $1/|Q^D|$ each. Then, the joint entropy of the k random variables $\mathbf{X}$ is $H(\mathbf{X}) = \log_2 |Q^D|$. Each subset of variables $X \subseteq \mathbf{X}$ defines a (marginal) distribution, with entropy $H(X)$. If $X = R_j$, then H must satisfy the following *cardinality constraint* $H(R_j) \leq \log_2 N_j = n_j$, because $\Pi_{R_j}(Q^D) \subseteq R_j^D$ and the marginal entropy is maximized when it is uniform. In addition, for any fd $U \rightarrow V$ the entropy must satisfy the *fd-constraint* $H(U) = H(UV)$.

We give here a very simple illustration of how the approach in [12] models the query output using entropy, by illustrating on the query $Q :- R(x, y), S(y, z), T(z, x)$ (with-out fd's) and output with five outcomes:

Q				R			S			T		
x	y	z		x	y		y	z		x	z	
a	3	r	$\frac{1}{5}$	a	3	$\frac{2}{5}$	3	r	$\frac{2}{5}$	a	r	$\frac{1}{5}$
a	2	q	$\frac{1}{5}$	a	2	$\frac{1}{5}$	2	q	$\frac{2}{5}$	a	q	$\frac{2}{5}$
b	2	q	$\frac{1}{5}$	b	2	$\frac{1}{5}$	3	q	$\frac{1}{5}$	b	q	$\frac{1}{5}$
d	3	r	$\frac{1}{5}$	d	3	$\frac{1}{5}$	4	q	0	d	r	$\frac{1}{5}$
a	3	q	$\frac{1}{5}$									

Here $H(xyz) = \log 5$, $H(xy) \leq \log |R| = \log 4$, $H(yz) \leq \log |S| = \log 4$, and $H(xz) \leq \log |T| = \log 4$.

GLVV Bound. Gottlob et al. [12] observe that $\log |Q^D| \leq \max_H H(\mathbf{X})$, where H ranges over all *entropic* functions that satisfy the cardinality constraints and the fd-constraints, and this bound has recently been shown to be tight [11]. However, we do not know how to compute this upper bound. The entropic cone requires an infinite number of (independent) constraints to characterize, and the precise characterization is a long-standing open problem in information theory [24]. To circumvent this difficulty, in [12] the entropic function H is replaced with a *polymatroid* function h . A *polymatroid* over a set of variables $\mathbf{X}$ is a function $h : 2^{\mathbf{X}} \rightarrow \mathbb{R}^+$ that satisfies the following inequalities, called the *Shannon inequalities*:

$$\begin{array}{ll} h(X) + h(Y) \geq h(X \cap Y) + h(X \cup Y) & \text{Sub-modularity} \\ h(X \cup Y) \geq h(X) & \text{Monotonicity} \end{array}$$

³ $\tilde{O}$ here hides log-factor in data complexity, and a polynomial factor in query size.

$$h(\emptyset) = 0$$

Zero

Every entropic function H is a polymatroid, and the converse fails when $\mathbf{X}$ has four or more variables [25]. Define $\text{GLVV}(Q, \text{FD}, (N_j)_j) \stackrel{\text{def}}{=} \max_h h(\mathbf{X})$, where h ranges over all polymatroids that satisfy all cardinality constraints $h(X_j) \leq n_j$, and all fd-constraints $h(UV) = h(U)$ for $U \rightarrow V \in \text{FD}$. Formally, Gottlob et al. [12] proved:

Theorem 2.2 (GLVV-bound). *Let Q be a join query with functional dependencies FD and input relations R_j of size $|R_j| = N_j$. Then, $|Q| \leq \text{GLVV}(Q, \text{FD}, (N_j)_{j=1}^m)$.*

We abbreviate the bound by $\text{GLVV}(Q)$ when FD and $(N_j)_{j=1}^m$ are clear from the context. Note the important fact that $\text{GLVV}(Q)$ can be computed by solving a linear program (with $2^k - 1$ variables). As we shall show, the bound is tight on normal lattices and some non-normal lattices we examined in this paper; but in general it is open whether the bound is always tight.

Closure. Fix a set FD . The *closure* of a set $X \subseteq \mathbf{X}$ is the smallest set X^+ such that $X \subseteq X^+$, and that $U \rightarrow V \in \text{FD}$ and $U \subseteq X^+$ imply $V \subseteq X^+$; X is *closed* if $X^+ = X$; the intersection of closed sets is closed. Denote by Q^+ the query obtained by replacing each relation $R_j(\text{vars}(R_j))$ with $R_j(\text{vars}(R_j)^+)$, then forgetting all functional dependencies; then a simple upper bound is $\max_D |Q^D| \leq \text{AGM}(Q^+)$. This bound is tight when all fd's are simple keys, because any product database over the schema of Q^+ can be converted into a database over the schema for Q that satisfies all simple keys. Thus, for simple keys, $\text{AGM}(Q^+)$ is a tight upper bound on $|Q^D|$. Theorem 4.4 in [12] proves essentially the same result. However, for fd's other than simple keys this technique fails, as illustrated by $Q(x, y, z) = R(x), S(y), T(x, y, z)$ where xy is a key in T (i.e. $xy \rightarrow z$): when $|R| = |S| = N$, $|T| = N^{100}$ we have $Q^+ = Q$ and $\text{AGM}(Q^+) = N^{100}$, yet one can easily verify that $|Q^D| \leq N^2$.

The Expansion Procedure. All our algorithms use the following simple pre-processing subroutine, called an expansion. Fix a relation $R(X)$ and a database instance D . R may be an input relation, or some intermediate relation generated during query evaluation. An *expansion* of R^D is some relation $(R^+)^D$ over attributes X^+ such that $\Pi_{X^+}(Q^D) \subseteq (R^+)^D$ and $\Pi_X((R^+)^D) \subseteq R^D$. If $X^+ = X$, then the expansion could be R^D itself, or any partial semi-join reduction that removes dangling tuples from R^D (which do not join with tuples in other relations). If $X \neq X^+$, then the expansion fills in the extra attributes, by repeatedly applying functional dependencies $X \rightarrow y$: if the fd is guarded in R_j , then it joins R with $\Pi_{X_j}(R_j)$; otherwise, if the fd corresponds to a UDF $y = f(X)$ then it simply computes f for each tuple in R . The expansion of R can be done in time $\tilde{O}(N)$ using trivial techniques.

3. A LATTICE-BASED APPROACH

3.1 Lattice representation of queries with FDs

Fix a query with functional dependencies (Q, FD) , over variables $\mathbf{X}$. It is well known [10, 14, 16] that the set of closed sets forms a lattice:

Definition 3.1 (Lattice representation). The lattice associated to FD is $\mathbf{L}_{\text{FD}} = (\mathbf{L}, \preceq)$, where $\mathbf{L}$ consists of all closed sets and the partial order $\preceq$ is $\subseteq$. The *lattice representation* of a query (Q, FD) is the pair $(\mathbf{L}_{\text{FD}}, \mathbf{R}^+)$, where

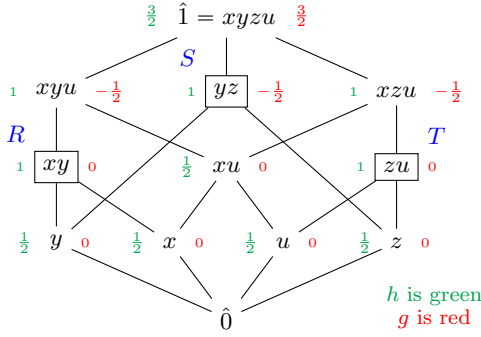


Figure 1: $Q := R(x, y), S(y, z), T(z, u), xz \rightarrow u, yu \rightarrow x$

$\mathbf{R}^+ = \{R_1^+, \dots, R_m^+\} \subseteq \mathbf{L}$ is the set of closures of the attributes of the input relations. We will use $\mathbf{L}$ to directly denote the lattice, with FD understood from context. With the expansion procedure in place, w.l.o.g. we assume that all the input R_j are closed sets; thus, the lattice representation of the query can be denoted simply by $(\mathbf{L}, \mathbf{R})$, where $\mathbf{R} = \{R_1, \dots, R_m\} \subseteq \mathbf{L}$, and $\bigvee \mathbf{R} = \bigvee_{R_j \in \mathbf{R}} R_j = \hat{1}$. (If $\bigvee \mathbf{R} < \hat{1}$, then some output variables are unbound!)

Note that the size of the lattice may be exponential in that of the query, but this does not affect our complexity results, because they are given in terms of data complexity only. If Q has no functional dependencies, then $\mathbf{L}$ is the Boolean algebra $2^{\mathbf{X}}$. We will use Fig. 1 as a running example, which illustrates the lattice for query (1). Lattice elements corresponding to input relations are framed.

We briefly review some notions of lattice theory needed later and refer the reader to [22] for an extensive treatment. Let $\wedge, \vee, \hat{0}, \hat{1}$ denote the greatest lower bound (*meet*), least upper bound (*join*), minimum and maximum elements of the lattice. The following hold in $\mathbf{L}_{\text{FD}}$: $X \wedge Y = X \cap Y$, $X \vee Y = (X \cup Y)^+$, $\hat{0} = \emptyset$, and $\hat{1} = \mathbf{X}$. We write $X \not\leq Y$ to mean X and Y are incomparable. An element U is said to *cover* an element V of $\mathbf{L}$ if $U \succ V$ and $U \succeq W \succeq V$ implies $W = U$ or $W = V$. A *chain* is a sequence of elements, one below the next in partial order. A *maximal chain* is a sequence of elements from $\hat{0}$ to $\hat{1}$ where one is covered by the next. An *atom* is an element X that covers $\hat{0}$; a *co-atom* is an element covered by $\hat{1}$. X is called a *join-irreducible* if $Y \vee Z = X$ implies $Y = X$ or $Z = X$. X is called a *meet-irreducible* if $Y \wedge Z = X$ implies $Y = X$ or $Z = X$. For any $X \in \mathbf{L}$, let $\Lambda_X \stackrel{\text{def}}{=} \{Z \mid Z \leq X, Z \text{ is a join-irreducible}\}$. The mapping $X \mapsto \Lambda_X$ defines a 1-1 mapping from $\mathbf{L}$ to the order ideals of the poset of join-irreducibles, with inverse $\Lambda_X \mapsto \bigvee \Lambda_X = X$.

Lattice representations and queries with FDs are in 1-to-1 correspondence, up to the addition/removal of variables functionally equivalent to some other set of variables. To see this in one direction, consider any pair $(\mathbf{L}, \mathbf{R})$ where $\mathbf{R} = \{R_1, \dots, R_m\} \subseteq \mathbf{L}$ and $\bigvee \mathbf{R} = \hat{1}$. Let $\mathbf{X}$ be the join-irreducibles of $\mathbf{L}$, and define a query (Q, FD) as follows. For $j \in [m]$, let R_j be a relation with attributes Λ_{R_j} . Define $\text{FD} \stackrel{\text{def}}{=} \{X \rightarrow \Lambda_{\bigvee X} \mid X \subseteq \mathbf{X}\}$, then the closed sets are $\{\Lambda_U \mid U \in \mathbf{L}\}$; thus, the lattice representation of this query is isomorphic to $(\mathbf{L}, \mathbf{R})$. In the other direction, consider a query with functional dependencies, (Q, FD) . Call a variable x redundant if $Y \leftrightarrow x$ for some set Y that does not

contain x . W.l.o.g. we can remove all redundant variables from Q (and decrease accordingly the arity of the relations containing x), because the values of a redundant variable can be recovered through expansion (Sec. 2). We claim $x \mapsto x^+$ is a 1-to-1 mapping between the variables of Q and the join-irreducibles of $\mathbf{L}_{\text{FD}}$. We first check that x^+ is join-irreducible: if $x^+ = Y \vee Z = (Y \cup Z)^+$ with $x \notin Y, x \notin Z$ then $x \leftrightarrow Y \cup Z$, contradicting the fact that x is not redundant. For injectivity, if $x \neq y$, then $x^+ = y^+$ implies $x \leftrightarrow y^+ - \{x\}$, again contradicting non-redundancy. Finally, surjectivity follows from the fact that $Y = \bigvee_{x \in Y} x^+$ for any closed set $Y \in \mathbf{L}$: if Y is join-irreducible then $Y = x^+$ for some $x \in Y$, proving surjectivity.

Thus, in the rest of the paper we shall freely switch between queries and lattices, using the following correspondence:

variable $x \in \mathbf{X}$	join-irreducible $X \in \mathbf{L}$
input $R_j(X_j)$	input $R_j \in \mathbf{L}$ with variables Λ_{R_j}
set of variables $X \subseteq \mathbf{X}$	set of join-irreducibles X
its closure X^+	$X^+ \stackrel{\text{def}}{=} \Lambda_Z$, where $Z = \bigvee X$
FD = $\emptyset$	Boolean Algebra

We prove the following in [2]:

Proposition 3.2. *If all functional dependencies are simple, then $\mathbf{L}$ is a distributive lattice.*

3.2 Database Instances for a Lattice

Let $\mathbf{X}$ denote the set of join-irreducibles of a lattice $\mathbf{L}$, and let $\text{FD} = \{X \rightarrow \Lambda_{\bigvee X} \mid X \subseteq \mathbf{X}\}$ be all fd's implicitly defined by $\mathbf{L}$. A *database instance* D for $\mathbf{L}$ is a relation with attributes $\mathbf{X}$ that satisfies FD. We denote $h_D : \mathbf{L} \rightarrow \mathbb{R}^+$ the restriction of its entropy function (as Sec. 2) to closed sets. If $(\mathbf{L}, \mathbf{R})$ is the lattice representation of a query (Q, FD) , then any database instance for $\mathbf{L}$ defines a standard database instance for Q by $R_j^D \stackrel{\text{def}}{=} \Pi_{R_j}(D)$. For example, the query

$$Q := R(x), S(y), T(z), xy \rightarrow z, xz \rightarrow y, yz \rightarrow x$$

has corresponding lattice $\mathbf{L}$ isomorphic to M_3 in Fig. 6. The relation $D = \{(i, j, k) \in [N]^3 \mid i + j + k \bmod N = 0\}$ is a database instance for M_3 , which yields the standard instance: $R^D = S^D = T^D = [N]$. Notice that in this example, all fd's are unguarded and hence lost in the standard instance. Starting from Sec. 5.1 we will assume that during the query evaluation we have access to the UDF's that defined the unguarded fd's.

Definition 3.3 (Materializability). A function $h : \mathbf{L} \rightarrow \mathbb{R}^+$ is called *materializable* w.r.t. an input query $(\mathbf{L}, \mathbf{R})$ if there exists an instance D such that $\log |R_j^D| \leq h(R_j)$ for all $j \in [m]$ and $\log |Q^D| \geq h(\hat{1})$.

3.3 The Lattice Linear Program

Given a query $(\mathbf{L}, \mathbf{R})$ and log cardinalities $(n_j)_{j=1}^m$, define the following linear program

$$\begin{aligned} & \text{maximize } h(\hat{1}) \\ & \forall X \not\leq Y : h(X \wedge Y) + h(X \vee Y) \leq h(X) + h(Y) \\ & \forall j \in [m] : h(R_j) \leq n_j \end{aligned} \tag{4}$$

over *non-negative* variables $(h(X))_{X \in \mathbf{L}}$. This is called the *Lattice Linear Program*, or LLP; its optimal objective will be shown to be exactly GLVV-bound.

A feasible solution h to LLP is called a (non-negative) **L-sub-modular** function. If h is also **L-monotone**, i.e. $X \preceq Y \Rightarrow h(X) \leq h(Y)$, and $h(\hat{0}) = 0$, then h is called an **L-polymatroid**. When **L** is clear from context we drop the **L**-prefix from **L-monotone**, **L-submodular**, and **L-polymatroid**, respectively. LLP did not require h to be a polymatroid because, at optimality, h^* can always be taken to be a polymatroid thanks to *Lovász's monotonization*: if h is non-negative **L-submodular**, then the function $\bar{h}(\hat{0}) \stackrel{\text{def}}{=} 0$ and $\bar{h}(X) \stackrel{\text{def}}{=} \min_{Y: X \preceq Y} h(Y)$, $X \neq \hat{0}$, is an **L-polymatroid** (see e.g. [21], pp. 774) and satisfies: $\bar{h}(\hat{1}) = h(\hat{1})$ and $\forall X, \bar{h}(X) \leq h(X)$. The full version [2] has the (very short) proof of the following lattice-based formulation of GLVV-bound:

Proposition 3.4. *Let h^* be an optimal solution to the LLP of a query $(Q, \text{FD}) = (\mathbf{L}, \mathbf{R})$. Then, $2^{h^*(\hat{1})} = \text{GLVV}(Q)$.*

When $\text{FD} = \emptyset$, **L** is a Boolean algebra in which case we show in [2] that $\text{AGM}(Q) = 2^{h^*(\hat{1})}$.

3.4 Embeddings

The main goal in this paper is to design algorithms that compute a query (Q, FD) (i.e. $(\mathbf{L}, \mathbf{R})$) in time $\tilde{O}(2^{h^*(\hat{1})})$. Whenever GLVV-bound is tight such an algorithm is worst-case optimal, hence our second goal is to study cases when the bound is tight. This happens if and only if the LLP has some optimal solution h^* that is materializable w.r.t. $(\mathbf{L}, \mathbf{R})$. A secondary goal in this paper is to find sufficient conditions for h to be materializable. Embeddings allow us to “transfer” materializability of polymatroids from one lattice to another.

Definition 3.5. An *embedding* between two lattices **L** and **L'** is a function $f : \mathbf{L} \rightarrow \mathbf{L}'$ such that $f(\bigvee X) = \bigvee f(X)$ for all $X \subseteq \mathbf{L}$, and $f(\hat{1}_{\mathbf{L}}) = \hat{1}_{\mathbf{L}'}$. An embedding between two queries $f : (\mathbf{L}, \mathbf{R}) \rightarrow (\mathbf{L}', \mathbf{R}')$ is an embedding from **L** to **L'** that is a bijection from **R** to **R'**.

If f is an embedding from **L** to **L'** and h' is a non-negative, sub-modular function on **L'**, then one can check that $h \stackrel{\text{def}}{=} h' \circ f$ is also sub-modular. If f is an embedding between queries, then their relation symbols are in 1-1 correspondence: $R_1, \dots, R_m$ and $R'_1, \dots, R'_m$ respectively. Fix two queries $(Q, \text{FD}), (Q', \text{FD}')$ with variables $\mathbf{X}, \mathbf{X}'$ respectively. Call a function $\mathcal{L} : 2^{\mathbf{X}} \rightarrow 2^{\mathbf{X}'}$ a *variable renaming* if $\mathcal{L}(X) = \mathcal{L}(X^+)$, $\mathcal{L}(X) = (\bigcup_{x \in X} \mathcal{L}(x))^+$ ($\mathcal{L}$ is uniquely defined by its values on single variables), and $\mathcal{L}(\mathbf{X}) = \mathbf{X}'$ (all variables in Q' are used). One can check that every embedding f defines the variable renaming $\mathcal{L}(X) \stackrel{\text{def}}{=} f(X^+)$ for all $X \subseteq \mathbf{X}$, and vice versa.

Fix an embedding $f : \mathbf{L} \rightarrow \mathbf{L}'$, and an instance D' for **L'**. Define the database instance $D \stackrel{\text{def}}{=} f^{-1}(D')$ as $D \stackrel{\text{def}}{=} \{t \circ \mathcal{L} \mid t \in D'\}$. In other words, every tuple $t' \in D'$ has attributes $\mathbf{X}'$, hence it can be viewed as a function $t' : \mathbf{X}' \rightarrow \text{Dom}$: for each such t' we include in D a tuple $t' \circ \mathcal{L} : \mathbf{X} \rightarrow \text{Dom}$, obtained from t' by renaming its attributes. One can check that $|D| = |D'|$ (because all variables in $\mathbf{X}'$ are used), and the same holds for all projections $|\Pi_X(D)| = |\Pi_X(D')|$ for all $X \subseteq \mathbf{X}$. This proves:

Proposition 3.6. *If $f : \mathbf{L} \rightarrow \mathbf{L}'$ is a lattice embedding, then for any instance D' for **L'**, $h_D = h_{D'} \circ f$, where $D \stackrel{\text{def}}{=} f^{-1}(D')$. If $f : (\mathbf{L}, \mathbf{R}) \rightarrow (\mathbf{L}', \mathbf{R}')$ is a query embedding,*

and $h' : \mathbf{L}' \rightarrow \mathbb{R}^+$ is materializable, then $h \stackrel{\text{def}}{=} h' \circ f$ is also materializable.

Definition 3.7. A *quasi-product database instance* for **L** is an instance of the form $f^{-1}(D')$ where f embeds **L** into a Boolean algebra **A**, and D' is a product instance for **A**.

In absence of fd's, the worst-case output size reaches the AGM-bound on some product instance. In Theorem 4.5 we characterize queries whose worst-case output size is the GLVV-bound, achieved on a quasi-product database instance.

3.5 Inequalities

In information theory, an *information inequality* is defined by a vector of real numbers $(w_X)_{X \subseteq \mathbf{X}}$ such that the inequality $\sum_{X \subseteq \mathbf{X}} w_X H(X) \geq 0$ holds for any entropic function H . Fix a query $(\mathbf{L}, \mathbf{R})$. An *output-inequality* is given by a vector of non-negative real numbers $(w_j)_{j \in [m]}$ such that the inequality

$$\sum_{j=1}^m w_j h(R_j) \geq h(\hat{1}) \quad (5)$$

holds for all polymatroids h on the lattice **L**. For example, in a Boolean algebra output inequalities correspond to Shearer's lemma: $\sum_j w_j h(R_j) \geq h(\mathbf{X})$ iff $(w_j)_{j=1}^m$ is a fractional edge cover of the hypergraph with nodes $\{x \mid x \in \mathbf{X}\}$ and hyperedges $\{R_j \mid j \in [m]\}$. Any output inequality gives us immediately an upper bound on the output size of a query, because (5) implies $h(\hat{1}) \leq \sum_j w_j n_j$ for any feasible solution h of the LLP with log-cardinalities $(n_j)_{j=1}^m$.

An output size upper bound is best if it is minimized, which is precisely the objective of the *dual-LLP*, which is defined over *non-negative* variables $(w_j)_{j=1}^m$ and $(s_{X,Y})_{X \preceq Y}$, corresponding to the input R_j and incomparable pairs of lattice elements (thus $s_{X,Y}$ is the same variable as $s_{Y,X}$). The definition of the dual-LLP and the proof of the following lemma can be found in the full version [2].

Lemma 3.8. *Let M denote the matrix of sub-modular inequalities (first inequality of the LLP (4)). Given a non-negative vector $(w_j)_{j \in [m]}$, define the vector $(c_X)_{X \in \mathbf{L}}$ by setting $c_{\hat{1}} \stackrel{\text{def}}{=} 1$, $c_{R_j} \stackrel{\text{def}}{=} -w_j$ and $c_X \stackrel{\text{def}}{=} 0$ otherwise. Then, the following statements are equivalent*

- (i) (5) holds for all polymatroids h
- (ii) (5) holds for all non-negative sub-modular functions h
- (iii) there exists $s \geq 0$ such that $c^T \leq s^T M$. Equivalently, w is part of a feasible solution (s, w) of the dual-LLP.

Furthermore, if (s^*, w^*) and h^* are dual- and primal-optimal solutions, then $h^*(\hat{1}) = \sum_{j=1}^m w_j^* n_j$.

4. NORMAL LATTICES

This section shows that an **L-polymatroid** h can be materialized as a quasi-product database instance iff it satisfies all output inequalities given by fractional edge covers of a certain hypergraph; in that case we call h *normal*. Normal polymatroids are the largest class of polymatroids that preserve the elegant properties of the AGM bound: upper bound given in terms of a fractional edge cover, and lower bound given by a (quasi-) product database. We then extend normality to lattices, which we call normal if the lattice's optimal polymatroid h^* is normal. We show that every feasible solution to LLP on a normal lattice is materializable, and thus the GLVV-bound is tight on a normal lattice.

4.1 Normal polymatroids

Recall the Möbius inversion formula [22] in a lattice $\mathbf{L}$:

$$h(X) = \sum_{Y: X \preceq Y} g(Y) \quad \text{iff} \quad g(X) = \sum_{Y: X \preceq Y} \mu(X, Y) h(Y) \quad (6)$$

where $\mu(X, Y)$ is the Möbius function on $\mathbf{L}$ [22]. In information theory, when h is an entropy and $\mathbf{L}$ a Boolean algebra, the quantity $-g(X)$ is exactly the (multivariate) *conditional mutual information* $I(\hat{1} - X \mid X)$, which we abbreviate CMI. There is a simple sufficient condition on g which implies that h , defined by Eq.(6) is a polymatroid. (See Appendix A or the full version [2] for missing proofs.)

Lemma 4.1. *Let g be any function s.t. $g(Z) \leq 0$ for $Z \prec \hat{1}$, and $g(\hat{1}) = -\sum_{Z \prec \hat{1}} g(Z)$. Then the function h defined by Eq.(6) is a polymatroid. Furthermore, if $\mathbf{L}$ is distributive, then h is a modular polymatroid, i.e. $h(X) + h(Y) = h(X \vee Y) + h(X \wedge Y)$ for all $X, Y \in \mathbf{L}$.*

Any function h satisfying the property stated in Lemma 4.1 is called a *normal submodular function*. If, furthermore, $g(Z) = 0$ for all $Z \prec \hat{1}$ other than the co-atoms, then we say that h is *strictly normal*. The function h in our running example Fig. 1 is strictly normal because it is defined by the CMI g shown in the figure. The function h on the left of Fig. 6 is not normal, because its CMI satisfies $g(\hat{0}) > 0$.

Normal polymatroids are precisely non-negative linear combinations of “step functions”. For every $Z \in \mathbf{L}$, the *step function* h_Z at Z is defined by $h_Z(X) = 1$ if $X \not\preceq Z$, and 0 otherwise. Every step function is normal, because its Möbius inverse is $g_Z(\hat{1}) = 1$, $g_Z(Z) = -1$, and $g_Z(X) = 0$ otherwise. Any non-negative linear combination of step functions is normal. Conversely, if h is a normal polymatroid, then denoting $a_Z = -g(Z)$ for all $Z \neq \hat{1}$ we have $a_Z \geq 0$ (since h is normal) and $g = \sum_Z a_Z g_Z$, implying $h = \sum_Z a_Z h_Z$.

There are a couple of interesting properties of normal polymatroids, both of which allow us to prove lower bounds for $h^*(\hat{1})$. The first property is a generalization of the fact that the AGM-bound is tight.

Lemma 4.2. *Let h be an integral, non-negative, submodular function h on a lattice $\mathbf{L}$. Then h is normal iff it is the entropy function of a quasi-product instance (Definition 3.7).*

The second property is a generalization of AGM-bound. To state the second property, we need:

Definition 4.3. Let $(\mathbf{L}, \mathbf{R})$ be a query given in lattice representation. The co-atomic hypergraph $H_{co} = (V_{co}, E_{co})$ is defined as follows. The nodes V_{co} are the co-atoms of $\mathbf{L}$, and $E_{co} = \{e_1, \dots, e_m\}$ with $e_j = \{Z \mid Z \in V_{co}, R_j \not\preceq Z, R_j \in \mathbf{R}\}$. In other words, each relation R_j defines a hyperedge e_j consisting of those nodes that do *not* contain the variables of R_j . A simple illustration of a co-atomic hypergraph is in Fig. 2.

Lemma 4.4. *Inequality (5) holds for all normal polymatroids h iff it holds for all strictly normal polymatroids iff $(w_j)_{j=1}^m$ is a fractional edge cover of H_{co} .*

The co-atomic hypergraph is the natural concept to capture Shearer’s lemma in a general lattice. In fact, every vector $(w_j)_{j=1}^m$ for which the output inequality (5) holds is a fractional edge cover of the co-atomic hypergraph. One may wonder whether the output inequalities could also be

described by the *atomic hypergraph*, defined in a similar way, however the latter does not seem to lead to any interesting properties, except for a Boolean algebra $2^{\mathbf{X}}$ where the atomic and co-atomic hypergraphs are isomorphic via $x \mapsto \mathbf{X} - \{x\}$.

4.2 Normal Lattices

A normal lattice is a lattice where, at optimality, the polymatroid is normal. This is captured by the following, which is the main result of this section. (See the full version [2] for a proof.)

Theorem 4.5. *Let $(\mathbf{L}, \mathbf{R})$ be a query in lattice representation. The following are equivalent:*

1. *For every non-negative, submodular function h there exists a normal polymatroid h' s.t. $h'(\hat{1}) = h(\hat{1})$ and $h'(R_j) \leq h(R_j)$ for all $R_j \in \mathbf{R}$.*
2. *The previous property holds for h' strictly normal.*
3. *Output inequality (5) holds for all non-negative, submodular functions h iff $\{w_j\}_{j=1}^m$ is a fractional edge cover of the co-atomic hypergraph.*
4. *Every non-negative submodular function h on $\mathbf{L}$ has a materialization that is a quasi-product database instance.*

We call $\mathbf{L}$ a *normal lattice w.r.t. $\mathbf{R}$* if it satisfies any of these conditions, and a *normal lattice* if it is normal w.r.t. all $\mathbf{R}$. Normal lattices appear to be a new concept. It is decidable whether $\mathbf{L}$ is normal w.r.t. $\mathbf{R}$ using a brute-force procedure. In a normal lattice the GLVV-bound is tight (by item 4); we show [2] that every distributive lattice is normal, and thus the GLVV-bound is tight on those lattices.

Example 4.6. The lattice $\mathbf{L}$ in Fig. 1 is normal w.r.t. inputs xy, yz, zu , which follows by exhaustively proving all inequalities defined by the fractional edge covers of the co-atomic hypergraph, shown in Fig. 2. For example, the edge cover $(1/2, 1/2, 1/2)$ corresponds to the inequality $h(xy) + h(yz) + h(zu) \geq 2h(\hat{1})$, which can be proven by $h(xy) + h(yz) \geq h(\hat{1}) + h(y)$ and $h(y) + h(zu) \geq h(\hat{1}) + h(0)$. In fact, $\mathbf{L}$ is normal w.r.t. any inputs. Notice that $\mathbf{L}$ is not distributive.

In the full version [2] we prove a necessary condition for normality, which we conjecture to be sufficient. Gottlob et al. [12] define a *coloring* of a query Q with variables $\mathbf{X}$ to be a function $\mathcal{L} : \mathbf{X} \rightarrow 2^{\mathbf{X}'}$ such that $\mathcal{L}(\mathbf{X}) \neq \emptyset$, and for any FD $X \rightarrow Y$, $\mathcal{L}(Y) \subseteq \mathcal{L}(X)$, where, for each set X , $\mathcal{L}(X) \stackrel{\text{def}}{=} \bigcup_{x \in X} \mathcal{L}(x)$. Then the *color number* of $\mathcal{L}$ is defined as $C(\mathcal{L}) = |\mathcal{L}(\mathbf{X})| / \max_j |\mathcal{L}(R_j)|$ and the *color number* of Q is $\max_{\mathcal{L}} C(\mathcal{L})$. They prove two results: if the functional dependencies are restricted to simple keys then $|Q^D| \leq (\max_j |R_j^D|)^{C(Q)}$, and moreover this bound is essentially tight, even for general functional dependencies.

Colorings correspond one-to-one to integral, normal polymatroids, via $h(x) = |\bigcup_{x \in X} \mathcal{L}(x)|$. To see this, in one direction let $\mathcal{L} : \mathbf{X} \rightarrow 2^{\mathbf{X}'}$ be a coloring and define $h(X) \stackrel{\text{def}}{=} |\bigcup_{x \in X} \mathcal{L}(x)|$. The function $f(X) \stackrel{\text{def}}{=} \bigcup_{x \in X} \mathcal{L}(x)$ is an embedding $f : \mathbf{L} \rightarrow 2^{\mathbf{X}'}$ into a Boolean algebra (we assume w.l.o.g. that $f(\mathbf{X}) = \mathbf{X}'$, otherwise we redefine $\mathbf{X}'$), and we have $h = h' \circ f$ where $h'(Y) \stackrel{\text{def}}{=} |Y|$: since h' is a strictly normal polymatroid Lemma A.1 says that h is also normal.

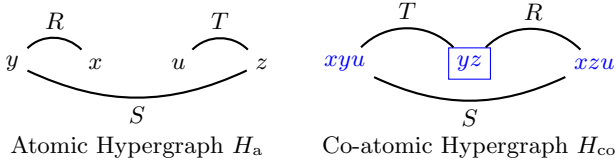


Figure 2: Atomic and Co-atomic hypergraphs for Fig.1

In the other direction, if h is an integral, normal polymatroid, then its canonical embedding (Definition A.2) defines a coloring $\mathcal{L}$ s.t. $h(X) \stackrel{\text{def}}{=} |\bigcup_{x \in X} \mathcal{L}(x)|$. Thus, colorings are essentially normal polymatroids. This implies the two results in [12] as follows. If FD consists only of simple keys, then the lattice $\mathbf{L}$ is distributive (Prop. 3.2), hence it is normal. Consequently, at optimality h^* can be assumed to be a normal polymatroid (Theorem 4.5 item 1), equivalently a coloring. The second result, tightness, follows from our Lemma 4.2, since any normal polymatroid (coloring) is the entropy of a quasi-product database instance.

5. PROOF SEQUENCE TO ALGORITHM

We aim to design an algorithm that, given an input query Q and database instance D , computes Q^D in time $\tilde{O}(2^{h^*(\hat{1})})$, where h^* is the optimal solution to the LLP (4). From Lemma 3.8, if (s^*, w^*) is dual-optimal then $h^*(\hat{1}) = \sum_{j=1}^m w_j^* n_j$ and the output inequality (5) holds with $w_j = w_j^*$, for all polymatroids. Our main algorithmic theme is to turn this process “inside-out”: find a “proof” of inequality (5) consisting of a sequence of steps transforming the left-hand-side into the right-hand-side, then interpret these steps as algorithmic steps whose runtime does not exceed what the symbolic step allows.

We show that three different proof techniques for (5) lead to three different bounds and algorithms. The first two are known techniques for proving Shearer’s lemma (i.e. the Boolean algebra case) that we adapt to lattices. The bounds are tight and the corresponding algorithms are worst-case optimal for some classes of lattices (such as distributive lattices, which include the Boolean algebra and the lattices of queries with simple fd’s, subsuming results from [12, 18, 23]). However, for a generic lattice the bounds these techniques can prove are not $h(\hat{1}) \leq \sum_j w_j^* h(R_j)$, and thus the corresponding algorithms’ runtimes in general can be worse than $\tilde{O}(2^{h^*(\hat{1})})$. An advantage of these two algorithms is that there is only one log-factor hidden in the $\tilde{O}$. Due to space limitations, the second bound and its corresponding algorithm are only described in the full version [2].

The last proof technique is our own, designed specifically to achieve the optimal LLP bound in an arbitrary lattice; this leads to an algorithm called CSMA running in the stated time of $\tilde{O}(2^{h^*(\hat{1})})$, where $\tilde{O}$ hides a polylogarithmic factor. This algorithm needs to regularize the data, somewhat similar in spirit to the uniformization step in [17] (see [2]).

5.1 Chain Bound and Chain Algorithm (CA)

Our first proof sequence adapts Jaikumar Radhakrishnan’s proof of Shearer’s lemma [20] to lattices, which is based on an arbitrary but fixed order of the random variables. We observe that a variable ordering corresponds to a maximal chain in the Boolean algebra. This idea allows us to gener-

alize Radhakrishnan’s proof to general lattices. Fix a chain $\mathbf{C} : \hat{0} = C_0 \prec C_1 \prec \dots \prec C_k = \hat{1}$ in $\mathbf{L}$. For $X \in \mathbf{L}$, we say that X covers i if $X \wedge C_i \neq X \wedge C_{i-1}$. Intuitively, X covers i if it contains some variable in C_i that does not appear in C_{i-1} . Fix a query represented by $(\mathbf{L}, \mathbf{R})$, where $\mathbf{R} = (R_1, \dots, R_m)$.

Definition 5.1. The *chain hypergraph* associated with a chain $\mathbf{C}$ is $H_{\mathbf{C}} = ([k], \{e_j \mid j \in [m]\})$, where the hyperedge e_j contains all nodes i such that R_j covers i .

In a Boolean algebra all maximal chains have the same hypergraph, which is the same as the query hypergraph, and the co-atomic hypergraph. But in a general lattice all these hypergraphs may be different, and some output inequalities (5) can be derived only from non-maximal chains. For that reason, we relax the maximality requirement on the chain, as follows. We say that the chain $\mathbf{C}$ is *good* for R_j if:

$$\text{for all } i \in [k] : i \in e_j \Rightarrow C_{i-1} \vee (R_j \wedge C_i) = C_i. \quad (7)$$

The key property of “goodness” is that submodularity applies in the following way (see [2]):

Proposition 5.2. If $\mathbf{C}$ is a maximal chain, then it is good for any R_j . Furthermore, if $\mathbf{C}$ is good for R_j , then, $\forall i \in e_j$,

$$h(R_j \wedge C_i) - h(R_j \wedge C_{i-1}) \geq h(C_i) - h(C_{i-1}), \quad (8)$$

for every $\mathbf{L}$ -submodular function h .

We say a chain $\mathbf{C}$ is *good* for $\mathbf{R}$ if it is good for all $R_j \in \mathbf{R}$. Radhakrishnan’s proof is adapted to a lattice as follows.

Theorem 5.3 (The Chain Bound). *Let $\mathbf{C}$ be any chain that is good for $\mathbf{R}$. If $(w_j)_{j=1}^m$ is any fractional edge cover of the chain hypergraph $H_{\mathbf{C}}$, then inequality (5) holds for any polymatroid h .*

Proof. By writing $h(R_j)$ as a telescoping sum, we obtain

$$\begin{aligned} \sum_{j=1}^m w_j h(R_j) &= \sum_{j=1}^m w_j \cdot \left(\sum_{i \in e_j} (h(R_j \wedge C_i) - h(R_j \wedge C_{i-1})) \right) \\ (\text{from Eq.(8)}) &\geq \sum_{j=1}^m w_j \cdot \left(\sum_{i \in e_j} (h(C_i) - h(C_{i-1})) \right) \\ &= \sum_{i=1}^k \left(\sum_{j: i \in e_j} w_j \right) \cdot (h(C_i) - h(C_{i-1})) \\ &\geq \sum_{i=1}^k (h(C_i) - h(C_{i-1})) \\ &= h(\hat{1}), \end{aligned}$$

where the last inequality holds because $(w_j)_{j=1}^m$ is a fractional edge cover of $H_{\mathbf{C}}$ and h is monotone. $\square$

Remark 5.4. Note that, if $\mathbf{C}$ is only good for a subset $\mathbf{R}'$ of $\mathbf{R}$, then we can apply the bound to $\mathbf{R}'$ with its corresponding chain hypergraph.

Example 5.5. Consider our running example from Figure 1; assume $|R| = |S| = |T| = N$. Consider the chain $\hat{0} \prec y \prec yz \prec xyzu = \hat{1}$, whose hypergraph has three vertices and hyperedges $e_R = \{y, xyzu\}$, $e_S = \{y, yz\}$, $e_T = \{yz, xyzu\}$. Thus, the chain bound on this chain is $N^{3/2}$,

which is tight. (Consider the input $R = S = T = [\sqrt{N}] \times [\sqrt{N}]$, where the fd $xz \rightarrow u$ is defined by the UDF $f(x, z) = x$ and $yu \rightarrow x$ by $g(y, u) = u$.)

If there were no FD's, then the chain bound is exactly Shearer's lemma [9] (or, equivalently, AGM bound).

Corollary 5.6 (AGM bound and Shearer's lemma). *Consider a join query on n variables with no fd's. The chain bound on the chain $\mathbf{C}$: $C_0 = \emptyset \prec C_1 = [1] \prec C_2 = [2] \prec \dots \prec C_n = [n]$ is exactly Shearer's lemma.*

Algorithm 1 The Chain Algorithm.

Input: A query $(\mathbf{L}, \mathbf{R})$, over $\mathbf{R} = \{R_1, \dots, R_m\}$
Input: A good chain $\mathbf{C} : \hat{0} = C_0 \prec C_1 \prec \dots \prec C_k = \hat{1}$
Input: Chain hypergraph $H_{\mathbf{C}} = ([k], \{e_1, \dots, e_m\})$
1: Expand Q to Q^+
2: $Q_0 = \{\emptyset\}$
3: **For** $i = 1$ **to** k **do**
4: $Q_i = \emptyset$
5: **For each** $t \in Q_{i-1}$ **do**
6: $T = \bigcap_{j:i \in e_j} (t \bowtie \Pi_{R_j \wedge C_i}(R_j))^+ \quad \triangleright$ Takes time
 $\tilde{O}\left(\min_{j:i \in e_j} |t \bowtie \Pi_{R_j \wedge C_i}(R_j)|\right)$. See text.
7: $Q_i = Q_i \cup T$
8: **Return** Q_k

The Chain Algorithm. In the proof above of the chain bound, the main idea is to take mixtures of conditional entropies (i.e. differences of two entropies), climbing up the chain. This strategy corresponds combinatorially to conditional search. From this proof, we derive Algorithm 1 that computes a query Q in time bounded by *any* fractional edge cover of the chain hypergraph. The algorithm assumes a fixed, good chain $\mathbf{C}$, where every node i is covered⁴. It pre-processes input relations by indexing them in an attribute order consistent with the chain. Then, it starts by expanding the query, as explained at the end of Sec. 2. Thus far it takes $\tilde{O}(N)$ -time. Next, the algorithm computes inductively

$$Q_i \stackrel{\text{def}}{=} \left(\bigwedge_{j:i \in e_j} \Pi_{R_j \wedge C_i}(R_j) \right)^+,$$

for $i = 0, 1, \dots, k$, where Q_i is an intermediate relation with attributes C_i . Initially Q_0 consists of just the empty tuple. Evidently, when $i = k$, Q_k is the output Q .

To compute Q_i , let R_j be some relation such that $i \in e_j$, in other words R_j has some new variable that occurs in C_i but not in C_{i-1} . Note that there exists at least one such R_j , because i is covered. Define

$$T_{ij} = Q_{i-1} \bowtie \Pi_{C_i \wedge R_j}(R_j),$$

whose attributes are $X_{ij} \stackrel{\text{def}}{=} C_{i-1} \cup (C_i \wedge R_j)$; and, by Eq.(7), its closure is $X_{ij}^+ = C_{i-1} \vee (C_i \wedge R_j) = C_i$. Consider T_{ij} 's expansion T_{ij}^+ (see Sec. 2): it has the same size as T_{ij} and has attributes $X_{ij}^+ = C_i$. Therefore,

$$Q_i = \bigcap_{j:i \in e_j} T_{ij}^+ = \bigcap_{j:i \in e_j} (Q_{i-1} \bowtie \Pi_{C_i \wedge R_j}(R_j))^+.$$

⁴If i is not covered, in other words if $H_{\mathbf{C}}$ has an isolated vertex, then $\rho^*(H_{\mathbf{C}}) = \infty$, and the algorithm will not work.

However, we do not want to compute all the T_{ij}^+ and then compute the intersection to obtain Q_i , because this naïve strategy will push the runtime over the budget. In order to stay within the time budget, the algorithm computes this intersection differently: it iterates over all tuples $t \in Q_{i-1}$, and for each tuple computes the intersection T in line 6 in time $\tilde{O}(\min_{j:i \in e_j} (|t \bowtie \Pi_{R_j \wedge C_i}(R_j)|))$. This can be accomplished by first computing $j_* = \text{argmin}_{j:i \in e_j} (|t \bowtie \Pi_{R_j \wedge C_i}(R_j)|)$ and tentatively setting $T = (t \bowtie \Pi_{R_{j_*} \wedge C_i}(R_{j_*}))^+$. Then, the algorithm removes from T any tuple t' that is not in the intersection defined in line 6. A tuple $t' \in T$ is not removed from T only if the following holds: for **every** $j \neq j_*$ s.t. R_j covers i , we have $\Pi_{C_i \wedge R_j}(t') \in \Pi_{C_i \wedge R_j}(R_j)$ and $(t \bowtie \Pi_{C_i \wedge R_j}(t'))^+ = t'$. Note the crucial fact that the relation R_{j_*} that is used to iterate over may depend on the tuple $t \in Q_{i-1}$. Due to the pre-processing step where every input relation is indexed with an attribute order consistent with the chain, j_* can easily be computed in logarithmic time in data complexity. The following is proved in Appendix B.

Theorem 5.7. *Assume the chain $\mathbf{C}$ is good for $\mathbf{R}$, and every node i is covered (i.e. no isolated vertices). Then, for any fractional edge cover of the chain hypergraph, $(w_j)_{j=1}^m$, the time taken by the Chain Algorithm to compute Q is $\tilde{O}(N + \prod_{j=1}^m N_j^{w_j})$, where $\tilde{O}$ hides a logarithmic factor needed for index lookup or binary search, and a small polynomial factor in query complexity.*

We present some examples.

Example 5.8. Continue with Example 5.5. The variable order corresponding to the chain $\hat{0} \prec y \prec yz \prec \hat{1}$ is $y, z, (xu)$, where x, u can be arranged in any order. The Chain Algorithm computes three intermediate relations:

$$\begin{aligned} Q_1(y) &= \Pi_y(R(xy)) \cap \Pi_y(S(yz)) \\ Q_2(yz) &= Q_1(y) \bowtie S(yz) \\ Q_3(yz xu) &= (Q_2(yz) \bowtie R(xy))^+ \cap (Q_2(yz) \bowtie T(zu))^+ \end{aligned}$$

The first two steps are straightforward. In the third, the join $Q_2(yz) \bowtie R(xy)$ results in a relation with attributes xyz , which needs to be expanded with u (e.g. by computing the user-defined function $u = f(x, z)$), and similarly for the second join. However, the algorithm does not compute the joins first then intersect, instead it iterates over tuples $t \in Q_2(yz)$ and computes an intersection on a per-tuple basis using the less expensive option. In particular, for each $t = (y, z) \in Q_2$, it compares $|t \bowtie R(x, y)|$ and $|t \bowtie T(z, u)|$ (which can be done in logarithmic time given that R was indexed with attribute order (y, x) and T with order (z, u)). Suppose $|t \bowtie R(x, y)| \leq |t \bowtie T(z, u)|$, then for each $(y, z, x) \in t \bowtie R(x, y)$ the algorithm uses the $xz \rightarrow u$ FD to obtain the tuple (y, z, x, u) . The next task is to verify that this tuple is indeed in the intersection as defined in line 6. This is done with *two* verifications: we make sure that $(z, u) \in T$, and that $yu \rightarrow x$ is indeed satisfied.⁵

On this chain the algorithm runs in optimal time $O(N^{3/2})$. We note that all previously proposed known worst-case optimal join algorithms for queries without FD's [18, 19, 23] require $\Omega(N^2)$ to compute the previous query on this instance: $R = S = T = \{(1, i) \mid i \in [N/2]\} \cup \{(i, 1) \mid i \in [N/2]\}$. For example, LFTJ with variable order y, z, x, u computes queries

⁵This is a subtle step in the algorithm that is easy to miss at the first read.

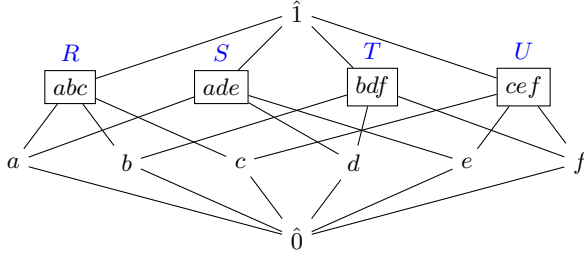


Figure 3: A query where the chain bound is not optimal

$Q_1(y), Q_2(yz), Q_3(xyz), Q_4(xyzu)$, where $|Q_3| = N^2$. Note, however, that not every maximal chain gives an optimal bound: for example the chain $\hat{0} \prec a \prec abc \prec \hat{1}$. Denoting $\{1, 2, 3\}$ the vertices of the chain hypergraph, its edges are $e_R = \{1, 2\}$, $e_S = \{1, 3\}$, $e_T = e_U = \{2, 3\}$. The optimal fractional edge cover is $(w_R, w_S, w_T, w_U) = (1/2, 1/2, 1/2, 0)$, $\rho^* = 3/2$, therefore the chain bound is $|Q^D| \leq N^{3/2}$ and the Chain Algorithm will run in this time. All other maximal chains give the same bound (non-maximal chains are not good). However, we show in [2] that $|Q^D| \leq N^{4/3}$, which means that no chain bound is tight.

Example 5.9 (Chain bound is not always tight). For some queries, even with normal lattices, no chains give a tight upper bound. This is illustrated by Q in Fig. 3. Consider the chain $\hat{0} \prec a \prec abc \prec \hat{1}$. Denoting $\{1, 2, 3\}$ the vertices of the chain hypergraph, its edges are $e_R = \{1, 2\}$, $e_S = \{1, 3\}$, $e_T = e_U = \{2, 3\}$. The optimal fractional edge cover is $(w_R, w_S, w_T, w_U) = (1/2, 1/2, 1/2, 0)$, $\rho^* = 3/2$, therefore the chain bound is $|Q^D| \leq N^{3/2}$ and the Chain Algorithm will run in this time. All other maximal chains give the same bound (non-maximal chains are not good). However, we show in [2] that $|Q^D| \leq N^{4/3}$, which means that no chain bound is tight.

Discussion. We show in [2] that the chain bound implies Shearer’s lemma and AGM bound from the Boolean algebra; more generally it is tight on all distributive lattices. The lattice corresponding to simple FDs is distributive, thus the chain algorithm is worst-case optimal for simple FDs. We also presented examples and proved results on how to choose a good chain so that the chain hypergraph does not have an isolated vertex.

For brevity, we have described the Chain Algorithm using breadth-first search (or bottom-up). It can also be adapted to a depth-first implementation (or top-down), as used for example in LogicBlox’ LFTJ [23], which does not materialize intermediate relations.

5.2 The CSMA Algorithm

There are lattices for which the chain bound and the SM-bound (for sub-modularity) [2] are not optimal.

Example 5.10. Consider the lattice in Fig. 4. It satisfies the inequality $h(M) + h(O) + h(P) \geq 2h(\hat{1}) + h(\hat{0})$, but the chain bound cannot derive it, and neither can the SM-proof [2]. It can be proved by summing up

$$h(M) + h(Z) \geq h(V) + h(G) \quad (9)$$

$$h(O) + h(Z) \geq h(W) + h(I) \quad (10)$$

$$h(P) + h(Z) \geq h(Y) + h(J) \quad (11)$$

$$h(V) + h(W) \geq h(\hat{1}) + h(R) \quad (12)$$

$$h(Y) + h(R) \geq h(\hat{1}) + h(Z) \quad (13)$$

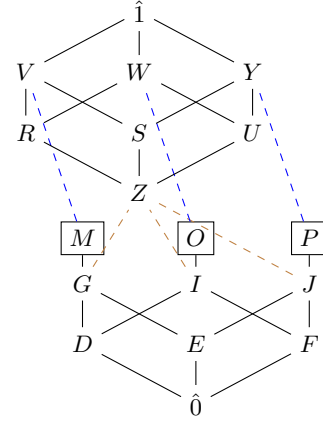


Figure 4: A lattice with a difficult-to-prove output inequality

$$h(G) + h(I) \geq h(Z) + h(D) \quad (14)$$

$$h(J) + h(D) \geq h(Z) + h(\hat{0}) \quad (15)$$

Our first goal in this section is to develop a set of rules that can prove any valid output inequality, including the one shown above in Example 5.10. In what follows, when $X \preceq Y$ we write $h(Y|X)$ as a short hand for $h(Y) - h(X)$. We assume h is a polymatroid. Our suggested proof strategy involves the following *three* basic rules:

- CD-rule: for $X \prec Y$, $h(Y) \rightarrow h(Y|X) + h(X)$.
- CC-rule: for $X \prec Y$, $h(Y|X) + h(X) \rightarrow h(Y)$.
- SM-rule: for $A \not\leq B$, $h(A) + h(B|A \wedge B) \rightarrow h(A \vee B)$.

(CD stands for *conditional decomposition*, CC *conditional composition*, and SM *sub-modularity*.) Together, they are called the *CSM rules* (for *conditional submodularity*). The vision behind CSMA is to show that

- every output inequality (5) can be proved by applying a series of CD, CC, and SM-rules: start with the left-hand side of an output inequality of the form (5), and repeatedly apply a CD/CC/SM-rule replacing the terms on the left-hand side of that rule with the terms on its right-hand side. In the end, we hope to end up with $h(\hat{1})$. The series of rules is then called a *CSM proof sequence* for the output inequality.
- each rule can be interpreted combinatorially to become an algorithmic step, and together they constitute a join algorithm (called CSMA) that runs in time $\tilde{O}(2^{h^*(\hat{1})})$.

Interestingly, we fell short of objective (A) yet were able to achieve a *stronger* result than objective (B)! In particular, with respect to (A) we conjecture that the three rules above form a complete proof system. We only managed to prove a weaker version of the conjecture (Theorem 5.16) which is sufficient for CSMA to work, at the price of a poly-log factor. On the other hand, with respect to (B) we will work with output inequalities that are more general than (5), and with constraints that are more general than the fd-constraints.

Appendix C contains running examples illustrating the bound and algorithm of this section.

Conditional LLP (CLLP). A key technical tool that helps realize the vision is a conditional version of LLP, called the *conditional LLP (CLLP)*. A nice by-product of CLLP is that we will be able to compute join queries with given degree bounds, which is even more general than join queries with functional dependencies!

Definition 5.11 (Degree). Let $X, Y \in \mathbf{L}$ be two lattice elements, and $T(X), T(Y)$ be two tables with attributes X, Y , respectively. Let $Z = X \wedge Y$ be the set of common variables in $T(X), T(Y)$. The $T(Y)$ -degree of some value $v \in \prod_{z \in Z} \text{Dom}(z)$ is the number of tuples in $T(Y)$ with $Z = v$:

$$\text{degree}_{T(Y)}(v) \stackrel{\text{def}}{=} |\sigma_{Z=v}(T(Y))|. \quad (16)$$

Definition 5.12 (CLLP). Let $\mathcal{P}$ be some set of pairs $(X, Y) \in \mathbf{L}^2$ such that $X \prec Y$. Assume for every pair $(X, Y) \in \mathcal{P}$, there is a given non-negative number $n_{Y|X}$ called a *log-degree bound*. The CLLP is defined as:

$$\begin{aligned} \max \quad & h(\hat{1}) \\ \text{s.t.} \quad & h(Y) - h(X) \leq n_{Y|X} \quad \forall (X, Y) \in \mathcal{P} \\ & h(A \vee B) + h(A \wedge B) \leq h(A) + h(B) \quad \forall A \not\leq B \in \mathbf{L} \\ & h(X) - h(Y) \leq 0 \quad \forall X \prec Y \in \mathbf{L}. \end{aligned}$$

(By default, $h(\hat{0}) = 0$, and $h(X) \geq 0$.) In other words, CLLP requires h to be a polymatroid, subject to the additional constraints $h(Y) - h(X) \leq n_{Y|X}, \forall (X, Y) \in \mathcal{P}$, which are called *log-degree constraints*.

Both cardinality constraints and fd constraints are special cases of the log-degree constraints. Intuitively, cardinality constraints are degree bounds of the empty tuple (i.e. $h(Y) - h(\hat{0}) \leq n_{Y|\hat{0}} = n_Y$). An fd-constraint $X \rightarrow Y$ imposes the Y -degree bound of 1 for every X -tuple (i.e. $n_{Y|X} = 0$). The following is obvious:

Proposition 5.13. LLP is exactly the same as CLLP for the special case when $\mathcal{P} = \{(\hat{0}, R_j) \mid R_j \in \mathbf{R}\}$.

Furthermore, we can easily use the log-degree constraints to encode input relations with known maximum degree bounds.

We will also need the dual-CLLP.

Definition 5.14 (Dual-CLLP). Let $c_{Y|X}, s_{A,B}$ and $m_{X,Y}$ denote the (non-negative) dual variables corresponding to the log-degree, sub-modularity, and monotonicity constraints, respectively. For each $Z \in \mathbf{L} - \{\hat{0}\}$, define

$$\begin{aligned} \text{netflow}(Z) \stackrel{\text{def}}{=} & \sum_{\substack{X: X \prec Z \\ (X, Z) \in \mathcal{P}}} c_{Z|X} - \sum_{\substack{Y: Z \prec Y \\ (Z, Y) \in \mathcal{P}}} c_{Y|Z} + \sum_{\substack{A \not\leq B \\ A \wedge B = Z}} s_{A,B} \\ & + \sum_{\substack{A \not\leq B \\ A \vee B = Z}} s_{A,B} - \sum_{A: A \not\leq Z} s_{A,Z} - \sum_{X: X \prec Z} m_{X,Z} + \sum_{Y: Z \prec Y} m_{Z,Y}. \end{aligned}$$

Then, the dual-CLLP is

$$\begin{aligned} \min \quad & \sum_{(X, Y) \in \mathcal{P}} n_{Y|X} c_{Y|X} \\ \text{s.t.} \quad & \text{netflow}(\hat{1}) \geq 1 \\ & \text{netflow}(Z) \geq 0 \quad \forall Z \in \mathbf{L} - \{\hat{1}, \hat{0}\}. \end{aligned} \quad (17)$$

CSM proof sequence. The analog of output inequality (5) in the conditional world is

$$\sum_{(X, Y) \in \mathcal{P}} c_{Y|X} h(Y|X) \geq h(\hat{1}) \quad (18)$$

Identical to Lemma 3.8, we can show that (18) holds for all polymatroids if there are vectors s and m such that (c, s, m) is feasible to the dual-CLLP.

To answer question (A), we prove a “reachability” lemma that helps us construct a CSM proof sequence. Let (c, s, m) be any feasible solution to the dual-CLLP (17). Let $K \subseteq \mathbf{L}$ be a set of lattice elements that contains $\hat{0}$. The *conditional closure* of K (with respect to (c, s, m)) is computed from K by repeatedly applying the following two steps: (1) CD-step (“Conditional Decomposition”): if $Y \in K$ and $X \prec Y$ then add X to K , (2) CC-step (“Conditional Composition”): if $X \in K$ and $c_{Y|X} > 0$ then add Y to K .

Lemma 5.15. For any dual-feasible solution (c, s, m) , let $K \subseteq \mathbf{L}$ be a set that contains $\hat{0}$, and $\bar{K}$ be its conditional closure. If $\hat{1} \notin \bar{K}$, then there are two lattice elements $A, B \in K$ such that $A \vee B \notin \bar{K}$ and $s_{A,B} > 0$.

Proof. If $\hat{1} \notin \bar{K}$, then $S \stackrel{\text{def}}{=} \sum_{Z \notin \bar{K}} \text{netflow}(Z) \geq 1$. If there is no such pair (A, B) , then every dual variable $c_{Y|X}, m_{X,Y}, s_{A,B}$ contributes a non-positive amount to the sum S , which is a contradiction. $\square$

The above lemma allows us to construct a “weak” CSM proof sequence for inequality (18), which is our answer to question (A) above. See Appendix C for a proof.

Theorem 5.16. Let (c, s, m) be an arbitrary feasible solution to the dual-CLLP, where $c_{Y|X} = q_{Y|X}/d$ and $s_{A,B} = q'_{A,B}/d$ are rational numbers. Let $\mathcal{B}$ be a multiset of variables $h(Y|X)$, $(X, Y) \in \mathcal{P}$, where each variable $h(Y|X)$ occurs $4^{|L|} \cdot q_{Y|X}$ times. Then, there is a sequence of CD-, CC-, and SM-rules⁶ that transforms $\mathcal{B}$ into another multiset $\bar{\mathcal{B}}$ which contains the variable $h(\hat{1})$. Moreover, in this sequence all occurrences of identical rules are consecutive (hence, they can be combined into a single “rule with a multiplicity”).

Interpreted integrally, we think of the CSM proof sequence described in Theorem 5.16 as having $D \stackrel{\text{def}}{=} 4^{|L|}d$ copies of $h(\hat{1})$ that it tries to reach, but at $\bar{\mathcal{B}}$ it reaches at least one copy and we stop. We call such a sequence *partial* in order to distinguish it from a (full) CSM proof sequence that reaches all D copies of $h(\hat{1})$.

The CSM algorithm (CSMA). CSMA is our answer to question (B). The algorithm takes as input a join query with functional dependencies and maximum degree bounds (if any) from input relations. This input is represented by the set $\mathcal{P}^{(0)}$, corresponding log-degree bounds, and the linear program CLLP⁽⁰⁾. For example, if we use CSMA for the original join query with functional dependencies (with no other max-degree bounds), then we would be starting with $\mathcal{P}^{(0)} = \{(\hat{0}, R_j) \mid R_j \in \mathbf{R}\}$; in this case CLLP⁽⁰⁾ is equivalent to LLP.

Let $h^{(0)}$ and $(c^{(0)}, s^{(0)}, m^{(0)})$ be a pair of primal and dual optimal solutions to CLLP⁽⁰⁾, and OPT be its optimal objective value. CSMA takes the (partial) CSM-proof sequence for CLLP⁽⁰⁾ as symbolic *instructions*. For each instruction, CSMA does some computation, spawning a number of sub-problems, creating new intermediate tables for the sub-problems if needed. The final output is contained in the union of outputs of the sub-problems.

⁶We can only apply SM-rules on pairs $A \not\leq B$ if $s_{A,B} > 0$.

For each sub-problem, CSMA constructs a new set of pairs $\mathcal{P}'$, and a new linear program CLLP' with dual-feasible solution (c', s', m') . Note that to construct CLLP' , there has to be a corresponding log-degree bound $n'_{Y|X}$ for each pair $(X, Y) \in \mathcal{P}'$. The following two invariants are maintained:

(Inv1) For any $(X, Y) \in \mathcal{P}'$, there is a table $T(Y)$ (a relation in the sub-problem) that “guards” the constraint $h(Y|X) \leq n'_{Y|X}$ of CLLP' in the sense that

$$\max_{v \in \Pi_X(T)} \log_2 \text{degree}_T(v) \leq n'_{Y|X}.$$

(Note that if $X = \hat{0}$ then the above says $\log_2 |T| \leq n'_{Y|\hat{0}} = n_Y$, which is a cardinality constraint.)

(Inv2) (c', s', m') is feasible to dual- CLLP' with objective value satisfying

$$\text{OBJ}' \stackrel{\text{def}}{=} \sum_{(X,Y) \in \mathcal{P}} n'_{Y|X} c'_{Y|X} \leq \text{OPT}.$$

It can be verified that the two invariants are satisfied at $\text{CLLP}^{(0)}$. Next we describe how CSMA deals with each instruction from the CSM sequence.

(1) CD-rule $h(Y) \rightarrow h(Y|X) + h(X)$ with multiplicity t .

Recall that $h(Y)$ is just a short hand for $h(Y|\hat{0})$. By (Inv1) there is a table $T(Y)$ with $\log_2 |T| \leq n_Y$. Lemma C.1 shows that T can be partitioned into at most $\ell = 2 \log N$ sub-tables $T^{(1)}, \dots, T^{(\ell)}$ such that $n_{Y|X}^{(j)} + n_X^{(j)} \leq n_Y$, for all $j \in [\ell]$, where

$$\begin{aligned} n_{Y|X}^{(j)} &\stackrel{\text{def}}{=} \max_{v \in \Pi_X(T^{(j)})} \log_2 \text{degree}_{T^{(j)}}(v), \\ n_X^{(j)} &\stackrel{\text{def}}{=} \log_2 |\Pi_X(T^{(j)})|. \end{aligned}$$

For each of these sub-tables $T^{(j)}$ of T , we create a sub-problem with T replaced by $T^{(j)}$. For the j th sub-problem, we add $(\hat{0}, X)$ and (X, Y) to $\mathcal{P}$ with corresponding log-degree constraints $h(X) \leq n_X^{(j)}$ and $h(Y|X) \leq n_{Y|X}^{(j)}$, respectively. We compute the projection of $T^{(j)}$ onto X so we have guards for the two new constraints. And we set $c'_{Y|X} = c_{Y|X} + t/D$, $c'_X = c_X + t/D$, $c'_Y = c_Y - t/D \geq 0$. If $c'_Y = 0$, then we remove $(\hat{0}, Y)$ from $\mathcal{P}$. By examining $\text{netflow}(Z)$ at each node, we can verify that (c', s, m) is a feasible solution to the new CLLP' with a *reduction* in objective value of $(n_Y - n_{Y|X}^{(j)} - n_X^{(j)})t/D$.

(2) CC-rule $h(Y|X) + h(X) \rightarrow h(Y)$ with multiplicity t .

Let R be the guard for $h(Y|X) \leq n_{Y|X}$ and S for $h(X) \leq n_X$. Let θ be a threshold to be determined. If $n_{Y|X} + n_X \leq \text{OPT} + \theta$, then we can compute the table $T(Y) \stackrel{\text{def}}{=} S(X) \bowtie R(Y)$ by going over all tuples in S and expanding them using matching tuples in R . The runtime is $\tilde{O}(2^{n_X + n_{Y|X}}) = \tilde{O}(2^{\text{OPT} + \theta})$. The dual solution is modified by setting

$$\begin{aligned} c'_{Y|X} &= c_{Y|X} - t/D, & c'_X &= c_X - t/D, \\ c'_Y &= c_Y + t/D, & n'_Y &= n_{Y|X} + n_X. \end{aligned}$$

If $n_{Y|X} + n_X > \text{OPT} + \theta$, we will start afresh from an optimal solution to the *current* CLLP . Lemma C.2 shows that the current CLLP has an optimal objective value at most $\text{OPT} - \theta/(D-1)$.

(3) SM-rule $h(A) + h(B|A \wedge B) \rightarrow h(A \vee B)$ with multiplicity t .

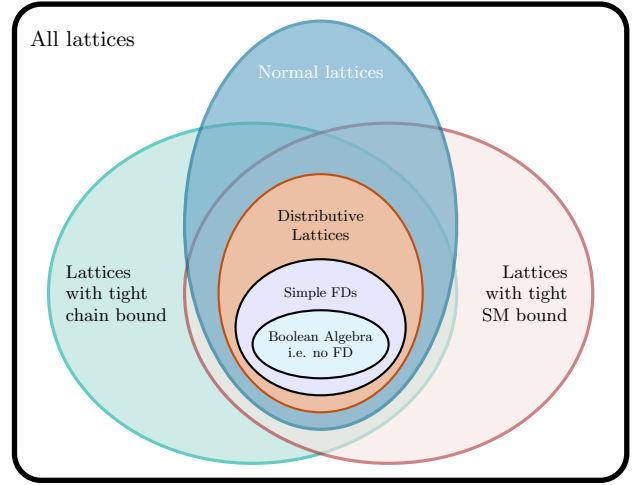


Figure 5: A summary of the lattices discussed in this paper

This is similar to case (2), and becomes identical to case (2) when $A = A \wedge B = X$ and $B = Y$. Let R be the guard for $h(A) \leq n_A$ and S for $h(B|A \wedge B) \leq n_{B|A \wedge B}$. If $n_A + n_{B|A \wedge B} \leq \text{OPT} + \theta$, then we can compute $T(A \vee B) = R \bowtie S$ in time $\tilde{O}(2^{\text{OPT} + \epsilon})$. If $n_A + n_{B|A \wedge B} > \text{OPT} + \theta$, then we start afresh from a new optimal solution to the current CLLP . The minor difference to case (2) is that we have to modify the variable $s'_{A,B} = s_{A,B} - t/D$. By selecting the correct threshold θ , we prove (in Appendix C) that

Theorem 5.17. *CSMA runs in time $\tilde{O}(N + (\log N)^e 2^{\text{OPT}})$, where OPT is the optimal objective value for $\text{CLLP}^{(0)}$, N is the input size, and e is a data-independent constant.*

6. CONCLUSIONS

We studied ways to prove worst-case output size bounds, and algorithms meeting the bounds for join queries with functional dependencies. A main aim was to design an algorithm running within time bounded by the entropy-based linear program proposed by Gottlob et al. [12]. For this purpose we developed new, lattice theoretic techniques, of independent interest. We described several classes of lattices, and several ways to prove upper or lower bounds on the worst-case query output, summarized in Fig. 5. On the algorithmic side, we devised the novel idea of turning a proof of an inequality into an algorithm. Three proof techniques lead to three different algorithms with increasing complexity. Our main algorithm does meet GLVV bound, up to a poly-log factor. The algorithm manages to solve a stronger problem, where input relations have prescribed maximum degree bounds, of which functional dependencies and cardinality bounds are special cases.

Acknowledgments

We thank Micheal Benedikt for thorough comments that significantly improved our presentation.

This work is partly supported by NSF grant 1535565, and by DARPA under agreement #FA8750-15-2-0009. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright thereon.

7. REFERENCES

- [1] C. R. Aberger, A. Nötzli, K. Olukotun, and C. Ré. Emptyheaded: Boolean algebra based graph processing. *CoRR*, abs/1503.02368, 2015.
- [2] M. Abo Khamis, H. Q. Ngo, and D. Suciu. Computing join queries with functional dependencies. *CoRR*, abs/1604.00111, 2016.
- [3] N. Alon, R. Yuster, and U. Zwick. Finding and counting given length cycles. *Algorithmica*, 17(3):209–223, 1997.
- [4] M. Aref, B. ten Cate, T. J. Green, B. Kimelfeld, D. Olteanu, E. Pasalic, T. L. Veldhuizen, and G. Washburn. Design and implementation of the logicblox system. In T. K. Sellis, S. B. Davidson, and Z. G. Ives, editors, *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31 - June 4, 2015*, pages 1371–1382. ACM, 2015.
- [5] A. Atserias, M. Grohe, and D. Marx. Size bounds and query plans for relational joins. In *49th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2008, October 25-28, 2008, Philadelphia, PA, USA*, pages 739–748, 2008.
- [6] P. Balister and B. Bollobás. Projections, entropy and sumsets. *Combinatorica*, 32(2):125–141, 2012.
- [7] M. Benedikt, J. Leblay, and E. Tsamoura. Querying with access patterns and integrity constraints. *PVLDB*, 8(6):690–701, 2015.
- [8] S. Chu, M. Balazinska, and D. Suciu. From theory to practice: Efficient join query evaluation in a parallel database system. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31 - June 4, 2015*, pages 63–78, 2015.
- [9] F. R. K. Chung, R. L. Graham, P. Frankl, and J. B. Shearer. Some intersection theorems for ordered sets and graphs. *J. Combin. Theory Ser. A*, 43(1):23–37, 1986.
- [10] J. Demetrovics, L. Libkin, and I. B. Muchnik. Functional dependencies in relational databases: A lattice point of view. *Discrete Applied Mathematics*, 40(2):155–185, 1992.
- [11] T. Gogacz and S. Toruńczyk. Entropy bounds for conjunctive queries with functional dependencies. *CoRR*, abs/1512.01808, 2015.
- [12] G. Gottlob, S. T. Lee, G. Valiant, and P. Valiant. Size and treewidth bounds for conjunctive queries. *J. ACM*, 59(3):16, 2012.
- [13] M. Grohe and D. Marx. Constraint solving via fractional edge covers. In *SODA*, pages 289–298. ACM Press, 2006.
- [14] P. Harremoës. Functional dependences and Bayesian networks. In *Proceedings WITMSE 2011*, Helsinki, 2011.
- [15] M. Joglekar and C. Ré. It’s all a matter of degree: Using degree information to optimize multiway joins. *CoRR*, abs/1508.01239, 2015.
- [16] M. Levene. A lattice view of functional dependencies in incomplete relations. *Acta Cybern.*, 12(2):181–207, 1995.
- [17] D. Marx. Tractable hypergraph properties for constraint satisfaction and conjunctive queries. *J. ACM*, 60(6):Art. 42, 51, 2013.
- [18] H. Q. Ngo, E. Porat, C. Ré, and A. Rudra. Worst-case optimal join algorithms: [extended abstract]. In *PODS*, pages 37–48, 2012.
- [19] H. Q. Ngo, C. Ré, and A. Rudra. Skew strikes back: new developments in the theory of join algorithms. *SIGMOD Record*, 42(4):5–16, 2013.
- [20] J. Radhakrishnan. 6. entropy and counting. *Computational Mathematics, Modelling and Algorithms*, page 146, 2003.
- [21] A. Schrijver. *Combinatorial optimization. Polyhedra and efficiency. Vol. B*, volume 24 of *Algorithms and Combinatorics*. Springer-Verlag, Berlin, 2003. Matroids, trees, stable sets, Chapters 39–69.
- [22] R. P. Stanley. *Enumerative combinatorics. Volume 1*, volume 49 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, Cambridge, second edition, 2012.
- [23] T. L. Veldhuizen. Triejoin: A simple, worst-case optimal join algorithm. In N. Schweikardt, V. Christophides, and V. Leroy, editors, *Proc. 17th International Conference on Database Theory (ICDT), Athens, Greece, March 24-28, 2014.*, pages 96–106. OpenProceedings.org, 2014.
- [24] R. W. Yeung. *Information Theory and Network Coding*. Springer Publishing Company, Incorporated, 1 edition, 2008.
- [25] Z. Zhang and R. W. Yeung. On characterization of entropy function via information inequalities. *IEEE Transactions on Information Theory*, 44(4):1440–1452, 1998.

APPENDIX

A. ADDITIONAL MATERIAL FOR SEC 4

Lemma A.1. *Let $f : \mathbf{L} \rightarrow \mathbf{L}'$ be an embedding between two lattices, and let h' be a normal polymatroid on $\mathbf{L}'$. Then $h \stackrel{\text{def}}{=} h' \circ f$ is a normal polymatroid on $\mathbf{L}$.*

Proof. Recall that an embedding is the left adjoint of a Galois connection. Let $r : \mathbf{L}' \rightarrow \mathbf{L}$ be its right adjoint⁷, in other words $f(X) \preceq Y$ iff $X \preceq r(Y)$. Let g' be the CMI for h' . The function $g(X) \stackrel{\text{def}}{=} \sum_{Y:r(Y)=X} g'(Y)$ is the CMI of h , because, for all $X \in \mathbf{L}$:

$$\begin{aligned}
 \sum_{Z:X \preceq Z} g(Z) &= \sum_{Z:X \preceq Z} \sum_{Z':r(Z')=Z} g'(Z') \\
 &= \sum_{Z':X \preceq r(Z')} g'(Z') = \sum_{Z':f(X) \preceq Z'} g'(Z') \\
 &= h'(f(X)) = h(X)
 \end{aligned}$$

In any Galois connection, $r(\hat{1}) = \hat{1}$ (since $f(\hat{1}) \preceq \hat{1}$ iff $\hat{1} \preceq r(\hat{1})$). By normality of h' we have $Y \neq \hat{1}$ implies $g'(Y) \leq 0$. Therefore, $X \neq \hat{1}$ implies $g(X) = \sum_{Y:r(Y)=X} g'(Y) \leq 0$, proving that h is normal. $\square$

We need the following to prove Lemma 4.2:

⁷The standard notation is g , but we use g for the CMI.

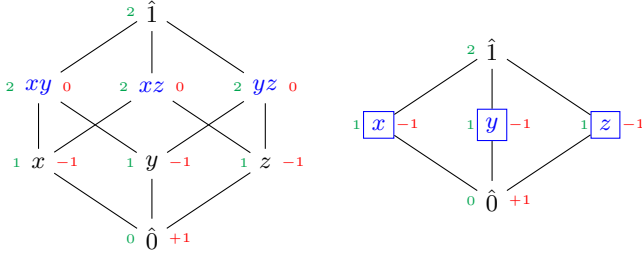


Figure 6: A non-normal function h (left) and the non-normal lattice M_3 (right)

Definition A.2. Let $A = 2^{\mathbf{X}}$ be a Boolean algebra. The canonical instance is the product database $D = [2]^{\mathbf{X}}$; note that $|\Pi_X(D)| = 2^{|X|}$, and thus $h_D(X) = |X|$, for all $X \subseteq \mathbf{X}$.

Let $\mathbf{L}$ be a lattice and h an integer-valued, normal polymatroid, and g its CMI. For all $X \in \mathbf{L}$, $X \neq \hat{1}$, let $C(X)$ be a set of $-g(X)$ arbitrary elements, such that the sets $(C(X))_{X \neq \hat{1}}$ are disjoint. Define $C(\hat{1}) = \emptyset$, and $\mathbf{C} \stackrel{\text{def}}{=} \bigcup_X C(X)$. The canonical Boolean Algebra is $A = (2^{\mathbf{C}}, \supseteq)$, and the canonical embedding of $\mathbf{L}$ is $f : \mathbf{L} \rightarrow A$, $f(X) \stackrel{\text{def}}{=} \bigcup_{Z: X \preceq Z} C(Z)$.

A is an “upside-down” Boolean algebra, where $Y \preceq Y'$ iff $Y \supseteq Y'$, and $\vee$ is set intersection; one can check that f commutes with $\vee$. We can now prove our result. (As in the AGM bound, there is a bit of loss when h are not integral, but this does not affect asymptotics)

Proof of Lemma 4.2. In one direction, let D be a quasi-product instance and h_D be its entropy function. By definition, $D = f^{-1}(D')$, where $f : \mathbf{L} \rightarrow A$ is an embedding into a Boolean algebra and D' is a product instance for A . Since $h_{D'}$ is normal (even strictly normal), h_D is also normal by Lemma A.1. In the opposite direction, assume h is normal. Let $f : \mathbf{L} \rightarrow A$ be the canonical embedding, and D' be the canonical instance for A . Then $D \stackrel{\text{def}}{=} f^{-1}(D')$ is the quasi-product instance, and one can check that $h = h_D = h_{D'} \circ f$. $\square$

Proof of Lemma 4.4. Since each (stricly) normal polymatroid is a non-negative linear combination of (co-atomic) step functions, it suffices to assume that h is a step function. First consider a co-atomic step functions h_Z , i.e. Z is a co-atom: the left-hand side of Eq. (5) is $\sum_{j: Z \in e_j} w_j$, and the right-hand side is 1, meaning that the inequality holds iff $(w_j)_{j=1}^m$ covers node $Z \in V_{\text{co}}$. It remains to show that if (5) holds for all co-atomic step functions, then it holds for all step functions h_X : let Z be any co-atom s.t. $Z \succeq X$, then $h_Z \leq h_X$ and $h_Z(\hat{1}) = h_X(\hat{1}) = 1$ implying that (5) holds for h_X . $\square$

Example A.3. The lattice on the left of Fig.6 is a Boolean algebra, hence it is normal: note that h in the figure is not normal⁸, but we can simply increase $h(\hat{1})$ to 3 and now it is normal. The lattice M_3 on the right of Fig.6 is not normal. Its co-atomic hypergraph has edges $e_x = \{y, z\}$, $e_y = \{x, z\}$, $e_z = \{x, y\}$ and the inequality corresponding to

⁸The function XOR on three variables, $R(x, y, z) = \{(a, b, c) \mid a, b, c \in \{0, 1\}, a \text{ xor } b \text{ xor } c = 0\}$, is the canonical example of a distribution whose entropy has a negative mutual information.

the fractional edge cover $(1/2, 1/2, 1/2)$, $h(x) + h(y) + h(z) \geq 2h(\hat{1})$, fails for the submodular function shown in the figure. In particular, this polymatroid h is not materializable as a quasi-product database instance, but can be materialized as $\{(i, j, k) \mid i, j, k \in \{0, 1\}, i + j + k = 0 \pmod{2}\}$.

B. ADDITIONAL MATERIAL FOR SEC 5.1

Proof of Theorem 5.7. We define some notation. For any tuple t and each relation R_j such that $R_j \wedge C_i \neq \hat{0}$, denote $n_{ijt} = |t \bowtie \Pi_{R_j \wedge C_i}(R_j)|$. Note that if $t = ()$ is the empty tuple, then $n_{ijt} = n_{ij()} = |\Pi_{R_j \wedge C_i}(R_j)|$.

Fix an iteration i . Denote by Time_i the time taken by iteration i (to compute Q_i). We will show that, for any $i \in [k]$,

$$\text{Time}_i = \tilde{O} \left(\prod_{j: R_j \wedge C_i \neq \hat{0}} n_{ij()}^{w_j} \right). \quad (19)$$

Since $n_{ij()} \leq |R_j|$, the sum over all Time_i will be at most $\tilde{O}(\prod_{j=1}^m |R_j|^{w_j})$, as desired. To bound Time_i , note that the number of steps taken to compute Q_i is

$$\sum_{t \in Q_{i-1}} \min_{j: t \in e_j} n_{ijt} \leq \sum_{t \in Q_{i-1}} \prod_{j: i \in e_j} n_{ijt}^{w_j} = \sum_{t \in Q_{i-1}} \prod_{j: R_j \wedge C_i \neq \hat{0}} n_{ijt}^{w_j}.$$

The equality follows from the fact that, if $t \in Q_{i-1}$ and $R_j \wedge C_i \neq \hat{0}$ but $i \notin e_j$, then $n_{ijt} = 1$. Hence, to show (19), it is sufficient to show the following:

$$\sum_{t \in Q_{i-1}} \prod_{j: R_j \wedge C_i \neq \hat{0}} n_{ijt}^{w_j} \leq \prod_{j: R_j \wedge C_i \neq \hat{0}} |n_{ij()}|^{w_j}. \quad (20)$$

For $0 \leq \ell \leq i-1$, define

$$T_\ell \stackrel{\text{def}}{=} \Pi_{C_\ell}(Q_{i-1}), \quad g(\ell) \stackrel{\text{def}}{=} \sum_{t \in T_\ell} \prod_{j: R_j \wedge C_i \neq \hat{0}} n_{ijt}^{w_j}.$$

By convention, T_0 has a single empty tuple $t = ()$. Then, (20) is equivalent to $g(i-1) \leq g(0)$. Thus, to show (20) it is sufficient to show that $g(\ell)$ is non-increasing in ℓ :

$$\begin{aligned} g(\ell) &= \sum_{t \in T_\ell} \prod_{j: R_j \wedge C_i \neq \hat{0}} n_{ijt}^{w_j} \\ &= \sum_{u \in T_{\ell-1}} \sum_{(u,v) \in T_\ell} \prod_{j: R_j \wedge C_i \neq \hat{0}} n_{ij(u,v)}^{w_j} \\ &= \sum_{u \in T_{\ell-1}} \prod_{j: R_j \wedge C_i \neq \hat{0}, i-1 \notin e_j} n_{ij u}^{w_j} \sum_{(u,v) \in T_\ell} \prod_{j: R_j \wedge C_i \neq \hat{0}, i-1 \in e_j} n_{ij(u,v)}^{w_j} \\ &\leq \sum_{u \in T_{\ell-1}} \prod_{j: R_j \wedge C_i \neq \hat{0}, i-1 \notin e_j} n_{ij u}^{w_j} \prod_{j: R_j \wedge C_i \neq \hat{0}, i-1 \in e_j} \left(\sum_{(u,v) \in T_\ell} n_{ij(u,v)} \right)^{w_j} \\ &\leq \sum_{u \in T_{\ell-1}} \prod_{j: R_j \wedge C_i \neq \hat{0}, i-1 \notin e_j} n_{ij u}^{w_j} \prod_{j: R_j \wedge C_i \neq \hat{0}, i-1 \in e_j} n_{ij u}^{w_j} \\ &= \sum_{u \in T_{\ell-1}} \prod_{j: R_j \wedge C_i \neq \hat{0}} n_{ij u}^{w_j} = g(\ell-1). \end{aligned}$$

The first inequality is generalized Hölder inequality, which applies because $(w_j)_{j=1}^m$ fractionally cover vertex $i-1$ of the chain hypergraph $H_{\mathbf{C}}$. The second inequality holds because, for every $u \in T_{\ell-1}$, $u \bowtie \Pi_{R_j \wedge C_i}(R_j) \supseteq T_\ell$. $\square$

C. ADDITIONAL MATERIAL FOR SEC. 5.2

Example (5.10 Continued: CLLP). Consider the lattice in Figure 4. Suppose the input relations are $T(M), T(O), T(P)$ (along with FDs imposing the lattice structure),⁹ and we are not given any other bounds on degrees/cardinalities, then the CLLP has $\mathcal{P} = \{(\hat{0}, M), (\hat{0}, O), (\hat{0}, P)\}$ where $n_M = n_{M|\hat{0}} = \log_2 |T(M)|$, $n_O = n_{O|\hat{0}} = \log_2 |T(O)|$, and $n_P = n_{P|\hat{0}} = \log_2 |T(P)|$. In this case CLLP is just LLP. If in addition we also knew, for example, an upper bound d on the degree of G in table $T(M)$, then we can extend the CLLP by adding (G, M) to $\mathcal{P}$ where $n_{M|G} = \log_2 d$.

As mentioned earlier, this lattice satisfies the inequality $2h(\hat{1}) \leq h(M) + h(O) + h(P)$, which does not admit an SM-proof. In the CLLP, this inequality corresponds to the constraint $h(\hat{1}) \leq \frac{n_M + n_O + n_P}{2}$, which results from the following dual solution.

$$\begin{aligned} c_M = c_O = c_P &= 1/2, \\ s_{M,Z} = s_{O,Z} = s_{P,Z} &= 1/2, \\ s_{V,W} = s_{R,Y} = s_{G,I} = s_{D,J} &= 1/2, \end{aligned} \quad (21)$$

(where all dual variables not specified above are zeros). In this dual solution, $\text{netflow}(\hat{1}) = 1$ and all other lattice nodes have a netflow value of 0. Hence, this solution is feasible.

Proof of Theorem 5.16. We prove the theorem with an algorithm. We start from the set $K = \{\hat{0}\}$, and keep adding elements to it using conditional closure (CC- and CD-) steps until K is conditionally closed; then we add a new element $A \vee B$ with an SM-step using the pair (A, B) found by Lemma 5.15. This process is repeated until $\hat{1} \in K$. With regard to $\mathcal{B}$, initially we will pretend that for each variable $h(Y|X) \in \mathcal{B}$ we have only $q_{Y|X}$ copies of it instead of $4^{|L|} q_{Y|X}$ copies. While adding elements to K , we will also add variables to $\mathcal{B}$ so eventually $\mathcal{B}$ contains $\leq 4^{|L|} q_{Y|X}$ of each variable.

We maintain the following invariants throughout the execution of the algorithm:

(1) For every $X \in K$, there is at least one copy of $h(X)$ in $\mathcal{B}$. To maintain this invariant, whenever we add a new element X to K while $\mathcal{B}$ does not contain any copy of $h(X)$, we apply some CSM rules on $\mathcal{B}$ to produce $h(X)$.

(2) For every copy of $h(Y|X)$ currently in $\mathcal{B}$, there will always remain at least one copy of $h(Y|X)$ after each step of the algorithm (i.e. in all subsequent multisets $\mathcal{B}$). To maintain this invariant, before applying any rule (where we will be losing one copy of each term on the left-hand side of that rule in order to gain one copy of each term on the right-hand side), we duplicate the multiplicities of all terms in the current $\mathcal{B}$, all previous multisets $\mathcal{B}$, and all rules that have been applied previously. (In effect, we re-run the entire history of rule application once.)

The above invariants are initially satisfied. Now we take the conditional closure of K . Whenever X is added to K due to some $Y \in K$ that satisfies $X \prec Y$ (signaling a CD-step), we first check whether $\mathcal{B}$ contains a copy of $h(X)$. If it does, then no further action is needed. If not, we duplicate multiplicities in the current and all previous $\mathcal{B}$ and all previous rules, and then we apply a CD-rule $h(Y) \rightarrow h(X) + h(Y|X)$.

⁹We abuse notation here, when the same notation symbol is used for different input relations. This is to avoid notation cluttering later in the description of the algorithm.

Whenever Y is added to K due to $c_{Y|X} > 0$ for some $X \in K$, we first check whether $h(Y)$ is in $\mathcal{B}$ already. If not, we duplicate all multiplicities as before, and then apply a CC-rule $h(X) + h(Y|X) \rightarrow h(Y)$.

Now, suppose K is already conditionally closed, we add $A \vee B$ using an SM-step guaranteed by Lemma 5.15. We check whether $h(A \vee B)$ is in $\mathcal{B}$. If it is not, we check whether $h(B|A \wedge B)$ is in $\mathcal{B}$. If it is not, we duplicate all multiplicities and apply a CD-rule $h(B) \rightarrow h(A \wedge B) + h(B|A \wedge B)$. Now, duplicate all multiplicities again, and apply an SM-rule $h(A) + h(B|A \wedge B) \rightarrow h(A \vee B)$.

For each element that is added to K , we have to duplicate its multiplicity at most twice, and there are at most $|\mathbf{L}|$ such elements.

Finally, we show that the same rule cannot be applied multiple times. In the above, before we applied any CC-rule that produced $h(Y)$, we checked whether $h(Y)$ was already in $\mathcal{B}$. Only if it was not, we applied the rule adding $h(Y)$ to $\mathcal{B}$, and letting the second invariant preserve it in $\mathcal{B}$. The same holds for SM-rules. In the above arguments, we applied CD-rules of the form $h(Y) \rightarrow h(X) + h(Y|X)$ in two different places: In the first, we checked that $h(X)$ was not in $\mathcal{B}$ before we added both $h(X)$ and $h(Y|X)$ to $\mathcal{B}$. In the second, we checked that $h(B|A \wedge B)$ was not in $\mathcal{B}$ before we added both $h(B|A \wedge B)$ and $h(A \wedge B)$ to $\mathcal{B}$. $\square$

Example (5.10 Continued: CSM proof sequence). Now, we simulate the proof of Theorem 5.16 on the dual solution in (21) (which corresponds to the inequality $h(\hat{1}) \leq \frac{h(M) + h(O) + h(P)}{2}$). We will start off with four copies of the right-hand side (i.e. $2h(M) + 2h(O) + 2h(P)$), and generate a CSM proof sequence that will produce one copy of $h(\hat{1})$ (out of 4 copies).

$$2h(M) \rightarrow 2h(M|G) + 2h(G) \quad (M \in \bar{K} \Rightarrow G \in \bar{K}) \quad (22)$$

$$2h(O) \rightarrow 2h(O|I) + 2h(I) \quad (O \in \bar{K} \Rightarrow I \in \bar{K}) \quad (23)$$

$$2h(I) \rightarrow 2h(I|D) + 2h(D) \quad (\text{extract } h(I|D)) \quad (24)$$

$$2h(G) + 2h(I|D) \rightarrow 2h(Z) \quad (\text{SM-rule, } s_{G,I} > 0) \quad (25)$$

$$h(Z) + h(M|G) \rightarrow h(V) \quad (\text{SM-rule, } s_{M,Z} > 0) \quad (26)$$

$$h(Z) + h(O|I) \rightarrow h(W) \quad (\text{SM-rule, } s_{O,Z} > 0) \quad (27)$$

$$h(W) \rightarrow h(W|R) + h(R) \quad (\text{extract } h(W|R)) \quad (28)$$

$$h(V) + h(W|R) \rightarrow h(\hat{1}) \quad (\text{SM-rule, } s_{V,W} > 0) \quad (29)$$

Note that rules (22)...(25) above had multiplicities of 2, because we needed to produce two copies of $h(Z)$: one for (26) and another for (27).

Example (5.10 Continued: CSMA). Consider a join query Q whose functional dependencies correspond to the lattice in Figure 4 and whose input relations are $T(M), T(O), T(P)$, all of size N . In the CLLP, we have $\mathcal{P} = \{(\hat{0}, M), (\hat{0}, O), (\hat{0}, P)\}$ where $n_M = n_O = n_P = (n \stackrel{\text{def}}{=} \log_2 N)$. The optimal objective value is $\text{OPT} = \frac{3n}{2}$ (which implies that $|Q| \leq N^{\frac{3}{2}}$), and the feasible dual solution given by (21) is optimal. Consider the CSM sequence (22)...(29) that was constructed earlier for (21). We will explain how to run CSMA on this sequence in order to answer Q in time within a polylogarithmic factor of $2^{\text{OPT}} = N^{\frac{3}{2}}$.

The first rule (22) in the sequence is a CD-rule: $h(M) \rightarrow h(M|G) + h(G)$. The corresponding algorithmic step would be to project $T(M)$ on G while making sure that the projection size times the maximum degree of the projection in

$T(M)$ does not exceed $|T(M)|$ (i.e. while making sure that the sum of $n'_G \stackrel{\text{def}}{=} \log_2 |\Pi_G(T(M))|$ and

$$n'_{M|G} \stackrel{\text{def}}{=} \max_{v \in \Pi_G(T(M))} \log_2 \text{degree}_{T(M)}(v)$$

does not exceed n_M). If all tuples v in the projection $\Pi_G(T(M))$ have the same degree $\text{degree}_{T(M)}(v)$ (i.e., if $T(M)$ is “uniform” with respect to G), then the required condition is met. Otherwise, let’s assume for simplicity that all degrees $\text{degree}_{T(M)}(v)$ are powers of 2 (and $|T(M)|$ is also a power of 2). If this is the case, then based on $\text{degree}_{T(M)}(v)$ we can partition $T(M)$ into a logarithmic number (namely $n_M + 1$) of parts $T^{(0)}(M), T^{(1)}(M), \dots$ such that the required condition is met in each one of them. (In particular, let $T^{(i)}(M)$ satisfy $n_{M|G}^{(i)} \leq i$ and $n_G^{(i)} \leq n_M - i$.) Now, the execution of CSMA will split into a logarithmic number of branches, each of which will continue on a different part of $T(M)$. For some arbitrarily-fixed i , let’s track the execution of the i -th branch (i.e. the one on $T^{(i)}(M)$).

The second rule (23) is another CD-rule: $h(O) \rightarrow h(O|I) + h(I)$. Similar to above, it will result in the partitioning of $T(O)$ into $O(\log N)$ parts $T^{(0)}(O), T^{(1)}(O), \dots$ (such that $T^{(j)}(O)$ satisfies $n_{O|I}^{(j)} \leq j$ and $n_I^{(j)} \leq n_O - j$.) The current i -th branch on $T^{(i)}(M)$ will now branch further into $O(\log N)$ branches corresponding to $T^{(0)}(O), T^{(1)}(O), \dots$. Let’s keep track of the j -th branch (on $T^{(j)}(O)$) for some arbitrary j .

The third rule (24) is yet another CD-rule: $h(I) \rightarrow h(I|D) + h(D)$. It will result in the partitioning of $T(I) \stackrel{\text{def}}{=} \Pi_I(T^{(j)}(O))$ (where $\Pi_I(T^{(j)}(O))$ resulted from the previous CD-step) into $O(\log N)$ parts: $T^{(0)}(I), T^{(1)}(I), \dots$ (such that $T^{(k)}(I)$ satisfies $n_{I|D}^{(k)} \leq k$ and $n_D^{(k)} \leq n_I - k$.) Let’s track the k -th branch (on $T^{(k)}(I)$).

The fourth rule (25) is an SM-rule: $h(G) + h(I|D) \rightarrow h(Z)$. The corresponding algorithmic step is to join the table $T(G)$ (or more precisely $\Pi_G(T^{(i)}(M))$ from Step 1) with the table $T(I)$ (or more precisely $T^{(k)}(I)$ from Step 3) in order to get a relation $T(Z)$. The time required to compute this join is bounded by the size of $T(G)$ times the maximum degree of D in $T(I)$. Depending on the current execution branch (i, j, k) , this time might or might not exceed our budget of $2^{\text{OPT}} = N^{3/2}$. For example, in the branch where both i and j are maximal, we will have $|T(G)| \leq 1$ and $|T(I)| \leq 1$, hence the join takes $O(1)$ time. On the other hand, in the branch where $i = j = 0$ and k is maximal, each one of $|T(G)|$, $|T(I)|$, and the maximum degree of D in $T(I)$ could be as large as N , hence the join could take time $O(N^2)$. Luckily, Lemma C.2 below implies that “when one door closes, another one opens”: In those particular branches where the join cannot be computed within our time budget, there are extra constraints that if considered in the CLLP they would reduce the optimal objective value OPT. For example, in the branch where $i = j = 0$ and k is maximal, the value of D is already fixed. This is because the degree of D in table $T(I)$ is maximal, hence it is equal to $|T(I)|$.

The remaining rules (26) ... (29) are similar to above.

Analysis of CSMA

Lemma C.1. *Let $X \prec Y$ be in the lattice $\mathbf{L}$. Let $T(Y)$ be a table with $\log_2 |T| \leq n_Y$. Then, T can be partitioned into at most $\ell = 2n_Y$ sub-tables $T^{(1)}, \dots, T^{(\ell)}$ such that $n_{Y|X}^{(j)} +$*

$n_X^{(j)} \leq n_Y$, for all $j \in [\ell]$, where

$$\begin{aligned} n_{Y|X}^{(j)} &\stackrel{\text{def}}{=} \max_{v \in \Pi_X(T^{(j)})} \log_2 \text{degree}_{T^{(j)}}(v) \\ n_X^{(j)} &\stackrel{\text{def}}{=} \log_2 |\Pi_X(T^{(j)})|. \end{aligned}$$

Proof. To obtain the copies $T^{(j)}$, observe that the number of elements $v \in \Pi_X(T)$ with log-degree in the interval $[j, j+1)$ is at most $|T|/2^j \leq 2^{n_Y-j}$. Hence, if we partition T based on which of the buckets $[j, j+1)$ the log-degree falls into, we would almost have the required inequality: $n_{Y|X}^{(j)} + n_X^{(j)} \leq (j+1) + (n_Y - j) = n_Y + 1$. To resolve the situation, we partition each $T^{(j)}$ into two equal-sized tables. Overall, we need $\ell = 2n_Y$. $\square$

Lemma C.2. *Given a CLLP whose optimal objective value is OPT, and a feasible dual solution (c, s, m) whose objective value is OBJ = $\sum_{(\bar{X}, \bar{Y}) \in \mathcal{P}} c_{\bar{Y}|\bar{X}} n_{\bar{Y}|\bar{X}}$, let $c_Y > 0$ for some $(\hat{0}, Y) \in \mathcal{P}$.*

- If $n_Y > \text{OBJ}$, then $\text{OPT} < \text{OBJ}$.
- Given $\theta \geq 0$, $0 < \epsilon < 1$, $\epsilon \leq c_Y$, if $n_Y > \text{OBJ} + \theta$, then $\text{OPT} < \text{OBJ} - \frac{\epsilon\theta}{1-\epsilon}$.

Proof. Let (c', s, m') be a dual solution obtained by setting $c'_Y = c_Y - \epsilon$, $m'_{Y,1} = m_{Y,1} + \epsilon$ and keeping other c' and m' values identical to their c and m counterparts. (c', s, m') is not necessarily feasible. However, it satisfies $\text{netflow}(\hat{1}) \geq 1 - \epsilon$ and $\text{netflow}(Z) \geq 0$ for all $Z \in \mathbf{L} - \{\hat{1}, \hat{0}\}$. Let (c'', s'', m'') be a dual solution obtained by multiplying (c', s, m') by $\frac{1}{1-\epsilon}$. Now (c'', s'', m'') is indeed feasible. Let OBJ'' be its objective value. Because it is dual feasible, $\text{OBJ}'' \geq \text{OPT}$.

$$\begin{aligned} \text{OBJ}'' &= \sum_{(\bar{X}, \bar{Y}) \in \mathcal{P}} c''_{\bar{Y}|\bar{X}} n_{\bar{Y}|\bar{X}} = \sum_{(\bar{X}, \bar{Y}) \in \mathcal{P}} \frac{c'_{\bar{Y}|\bar{X}}}{1-\epsilon} n_{\bar{Y}|\bar{X}} \\ &= \frac{1}{1-\epsilon} \sum_{(\bar{X}, \bar{Y}) \in \mathcal{P}} c_{\bar{Y}|\bar{X}} n_{\bar{Y}|\bar{X}} - \frac{\epsilon}{1-\epsilon} n_Y \\ &< \frac{1}{1-\epsilon} \text{OBJ} - \frac{\epsilon}{1-\epsilon} (\text{OBJ} + \theta) \\ &= \text{OBJ} - \frac{\epsilon\theta}{1-\epsilon}. \end{aligned}$$

$\square$

Proof of Theorem 5.17. The number of CD-rules is at most $|\mathbf{L}|^2$ (these are rules with multiplicities, so no rule is repeated). The worst case is obtained when the algorithm branches as far as possible only to reduce the objective by $\epsilon = \frac{\theta}{D-1}$ at the very end and all leaf nodes of the execution tree have to be started afresh with the new optimal value. Let $x = 2^{\text{OPT}}$ and $T(x)$ denote the runtime of the algorithm on the CLLP⁽⁰⁾ with objective value OPT. Then, the recurrence for the runtime is

$$\begin{aligned} T(x) &= (\ell)^{|\mathbf{L}|^2} T\left(2^{\text{OPT} - \frac{\theta}{D-1}}\right) + \ell^{|\mathbf{L}|^2} 2^\theta x \\ &= (\ell)^{|\mathbf{L}|^2} T\left(\frac{x}{2^{\theta/(D-1)}}\right) + \ell^{|\mathbf{L}|^2} 2^\theta x \end{aligned}$$

To get the exponential decay effect we set θ so that $2^{\theta/(D-1)} = 2\ell^{|\mathbf{L}|^2}$, which means $\theta = (D-1)(|\mathbf{L}|^2 \log_2 \ell + 1)$. $\square$